

Détection Prédictive de Fraude Bancaire: Une approche de Machine Learning

ABDOULAYE TANGARA

Student in MSc in Quantitative and Computable Economics

Contacts

 [LinkedIn](#)

 [Mon GitHub](#)

 [Mon Portfolio](#)

 abdoulayetangara722@gmail.com



"To get something you never had, you've got to do something you never did. Get your mind right"

Contexte :

Il y a quelques années, un de mes cousins a été victime d'une fraude bancaire. Un matin, il consulte son relevé de compte et découvre que plusieurs virements ont été effectués à son insu durant la nuit. Des sommes modestes, transférées vers des comptes inconnus, mais cumulées, elles représentaient presque tout son solde.

Il avait pourtant activé les alertes SMS et n'avait cliqué sur aucun lien suspect. Après enquête, la banque conclut à un piratage silencieux, utilisant des transactions éclatées et discrètes pour contourner les systèmes de détection.

Cet événement a été un déclic. Il m'a fait prendre conscience que la fraude moderne ne se manifeste plus toujours par des montants exorbitants ou des comportements flagrants. Elle devient sournoise, évolutive, algorithmique.

Ce projet est donc né de cette prise de conscience : construire un modèle prédictif basé sur l'IA, capable de repérer les signaux faibles que l'humain ne perçoit pas, et d'alerter avant qu'il ne soit trop tard.

Problématique :

Comment exploiter la puissance du Machine Learning pour identifier des schémas frauduleux complexes, tout en minimisant les faux positifs qui perturbent l'expérience client ?

Objectif :

Ce projet vise à développer un modèle prédictif capable de détecter les transactions frauduleuses en temps réel, en s'appuyant sur une analyse approfondie des données transactionnelles (montants, soldes, types d'opérations). Grâce à des techniques avancées de feature engineering et à l'algorithme XGBoost, le système parvient à une précision de 99 %, tout en maintenant un taux de rappel élevé pour ne pas laisser passer les fraudes.

Enjeux :

- Réduire les pertes financières liées aux fraudes.
- Améliorer la confiance des clients dans les transactions digitales.
- Automatiser la détection pour une réponse plus rapide que les méthodes manuelles.

Importation des modules nécessaires

```
[1]: # Chargement des modules neccessaires
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt #
import seaborn as sns
import plotly.express as px
import scipy.stats as stat
from tabulate import tabulate
from scipy import stats

from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, confusion_matrix

import shap
import warnings
warnings.filterwarnings('ignore')
pd.options.mode.chained_assignment = None
```

Gestion du temps

```
[2]: # Définition des tâches et durées
tache = {
    "Tâches": [
        "Etude du contexte",
        "Recherche & Traitement des données",
        "Analyse exploratoire des Données",
        "Modélisation ML",
        "Tests & validation",
        "Analyse des résultats",
        "Rédaction & Publication"
    ],
    "Début": ["2025-03-17", "2025-03-25", "2025-04-02", "2025-04-11", "2025-04-14", "2025-04-17", "2025-04-22"],
    "Fin": ["2025-03-23", "2025-03-31", "2025-04-10", "2025-04-13", "2025-04-17", "2025-04-21", "2025-05-01"]
}

# Tracer le diagramme de Gantt
df = pd.DataFrame(tache)
title = "Diagramme de GANTT"
axis_title = "Dates d'exécution (semaines)"
yaxis_title = "Tâches à réaliser"
```

```
def gantt_diagram (df, title=None, xaxis_title=None, yaxis_title=None):

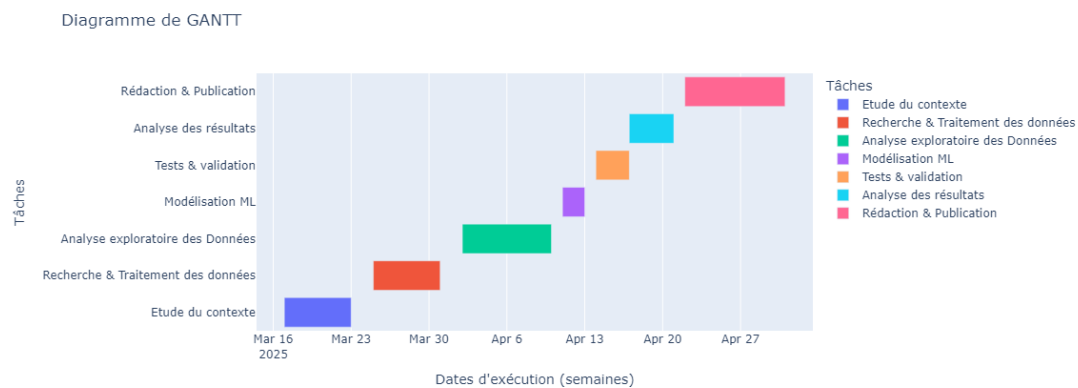
    """

    Il s'agit d'une fonction qui permet de générer un Diagramme de GANTT
    montrant l'agencement du projet.

    """

    fig = px.timeline(df, x_start="Début", x_end="Fin", y="Tâches",
    ↪color="Tâches",
        title=title, labels={"Task": yaxis_title})
    fig.update_yaxes(autorange="reversed")
    fig.update_layout(xaxis_title=xaxis_title)
    fig.show()

    # Création du Gantt chart
    gantt_diagram(df, title=title, xaxis_title=xaxis_title,
    ↪yaxis_title=yaxis_title)
```



0. Chargement des données

```
[3]: # Chargement du dataset
data = pd.read_csv("fraud_data.csv", delimiter = ',')
df_5_row = data.head()
df_5_row
```

```
[3]:
```

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	\
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	

	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	M1979787155	0.0	0.0	0	0

1	M2044282225	0.0	0.0	0	0
2	C553264065	0.0	0.0	1	0
3	C38997010	21182.0	0.0	1	0
4	M1230701703	0.0	0.0	0	0

```
[4]: # Information sur le dataset
print(f"Le nombre de clients : {data.shape[0]} \n")
print(f"Le nombre de caractéristiques : {data.shape[1]} \n")
```

Le nombre de clients : 6362620

Le nombre de caractéristiques : 11

1. Analyse Univarée

```
[5]: class AnalyseUnivariee:
    def __init__(self, df):
        self.df = df
        self.var_quanti = df.select_dtypes(include=[float]).columns.tolist()
        self.var_quali = df.select_dtypes(include=[int, object, "category"]).
→columns.tolist()

        # On enlève les colonnes non pertinentes si elles existent
        for col in ["nameOrig", "step", "nameDest"]:
            if col in self.var_quali:
                self.var_quali.remove(col)

    def analyser_quanti(self):
        print(f"\nStatistique descriptive des variables quantitatives :
→\n{self.df[self.var_quanti].describe()}\n")

        print("Analyse de la dispersion (boxplots) :")
        for col in self.var_quanti:
            plt.figure(figsize=(10, 5))
            sns.boxplot(x=self.df[col])
            plt.title(f"Boxplot de {col}")
            plt.xlabel(col)
            plt.show()

        print("Analyse de la normalité (histogrammes + Shapiro) :")
        for col in self.var_quanti:
            sw, pvalue = stat.shapiro(self.df[col].dropna()) # on évite les
→NaN

            plt.figure(figsize=(10, 5))
            sns.histplot(self.df[col], kde=True)
            if pvalue < 0.05:
                plt.title(f"{col} n'est pas normalement distribuée (p = {np.
→round(pvalue, 2)})")
            else:
                plt.title(f"{col} est normalement distribuée (p = {np.
→round(pvalue, 2)})")
```

```

plt.show()

def analyser_quali(self):
    print(f"\nStatistique descriptive des variables qualitatives :\n{self.
→df[self.var_quali].describe(include='all')}\n")

    for col in self.var_quali:
        counts = self.df[col].value_counts()
        plt.figure(figsize=(13, 8))
        plt.pie(counts, labels=counts.index, autopct='%1.1f%%',
→startangle=90,
                colors=['skyblue', 'salmon', 'lightgreen'])
        plt.title(f"Répartition de {col}")
        plt.show()

def lancer_analyse(self):
    print("\nDÉBUT DE L'ANALYSE UNIVARIÉE\n")
    if self.var_quanti:
        self.analyser_quanti()
    if self.var_quali:
        self.analyser_quali()
    print("\n FIN DE L'ANALYSE UNIVARIÉE\n")

```

```

[ ]: # Resultats des analyses
if __name__ == "__main__":
    print("\nDÉBUT DE L'ANALYSE UNIVARIÉE\n")
    # Instanciation de la classe AnalyseUnivariee
    # et lancement de l'analyse univariee
    analyse_1 = AnalyseUnivariee(data)

    # Analyse des variables quantitative
    analyse_1.analyser_quanti()

    # Analyses des variables qualitatives
    analyse_1.analyser_quali()

```

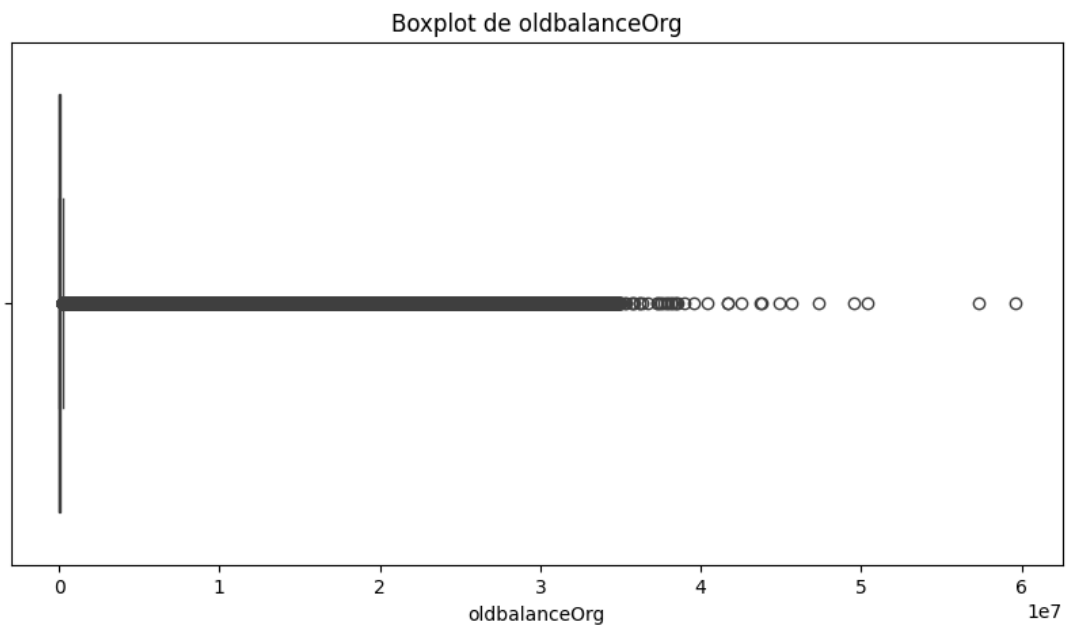
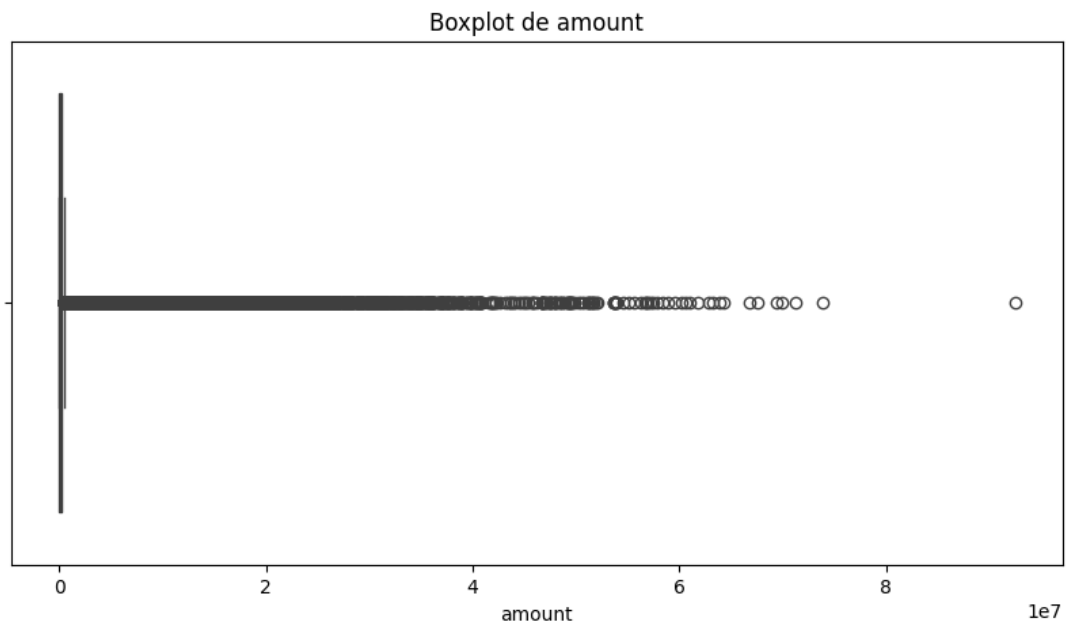
Statistique descriptive des variables quantitatives :

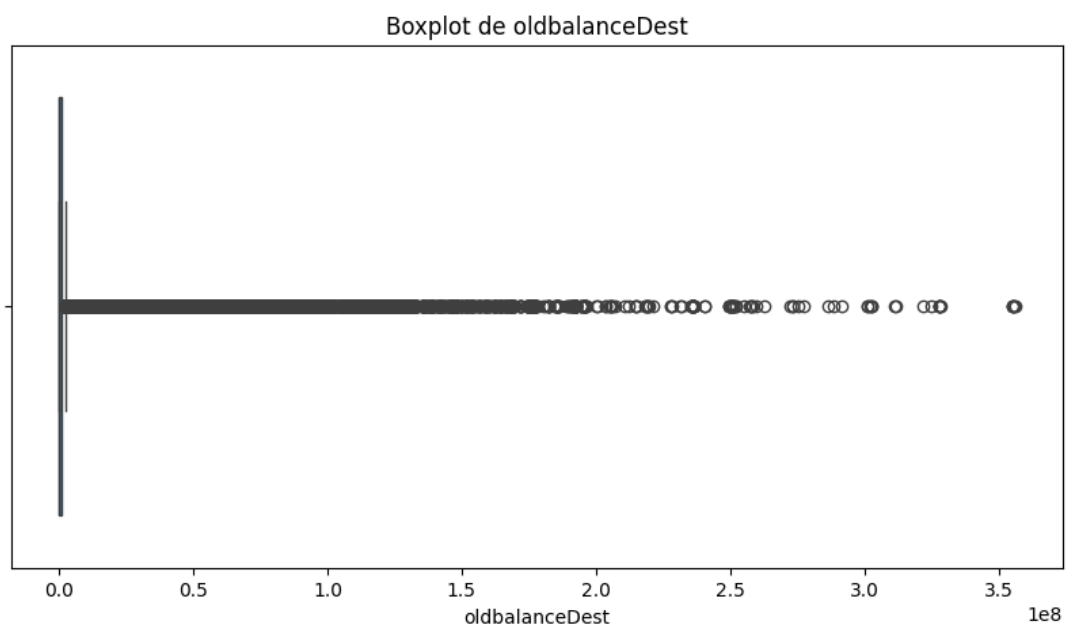
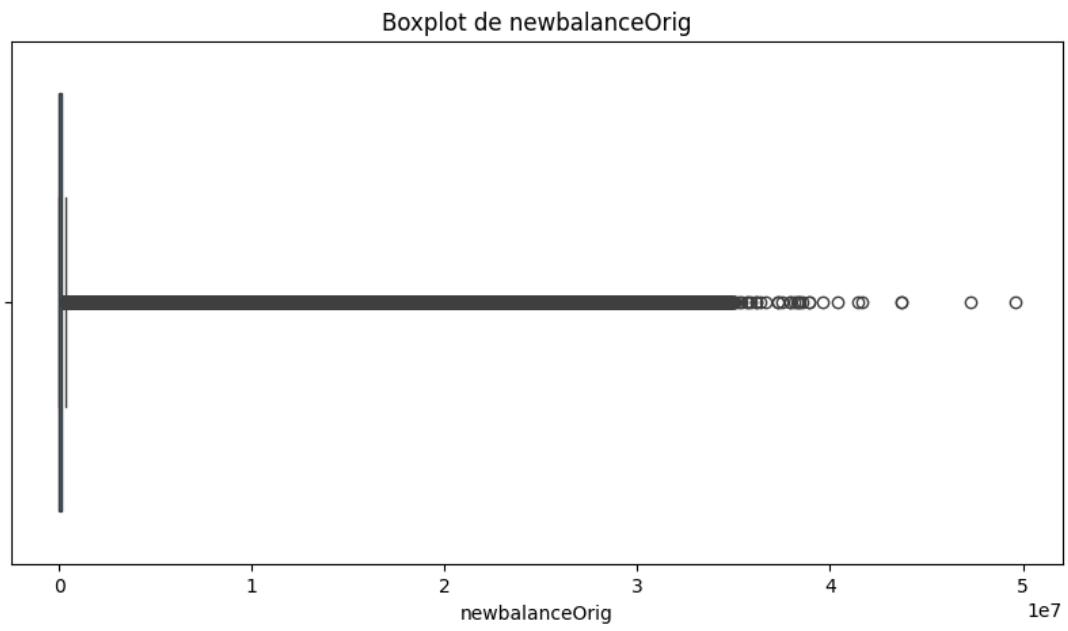
	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest \
count	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06
mean	1.798619e+05	8.338831e+05	8.551137e+05	1.100702e+06
std	6.038582e+05	2.888243e+06	2.924049e+06	3.399180e+06
min	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	1.338957e+04	0.000000e+00	0.000000e+00	0.000000e+00
50%	7.487194e+04	1.420800e+04	0.000000e+00	1.327057e+05
75%	2.087215e+05	1.073152e+05	1.442584e+05	9.430367e+05
max	9.244552e+07	5.958504e+07	4.958504e+07	3.560159e+08

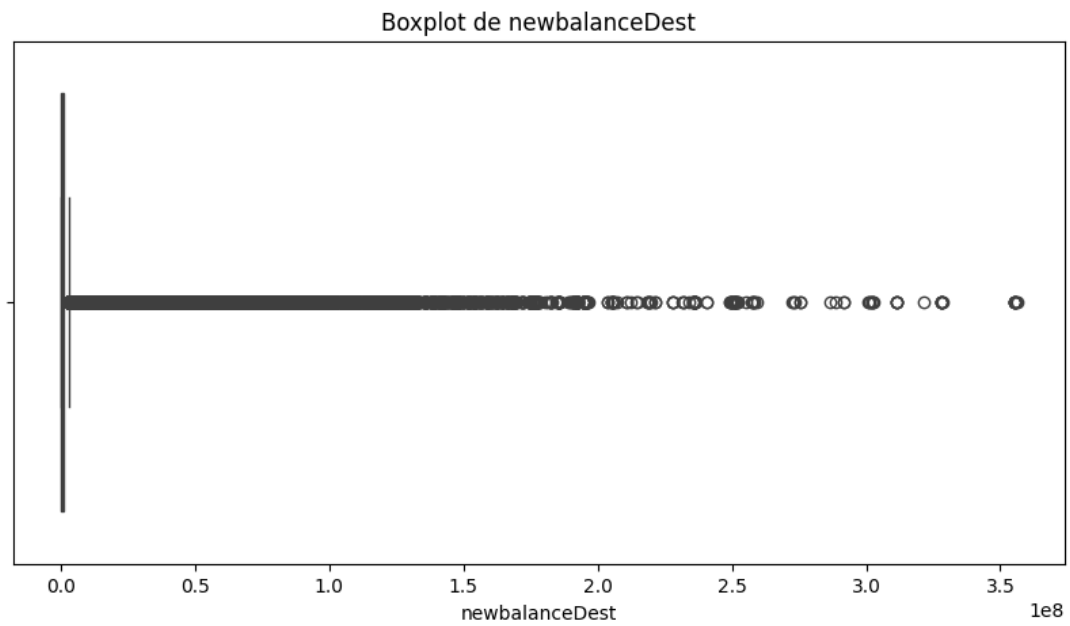
	newbalanceDest
count	6.362620e+06
mean	1.224996e+06

std	3.674129e+06
min	0.000000e+00
25%	0.000000e+00
50%	2.146614e+05
75%	1.111909e+06
max	3.561793e+08

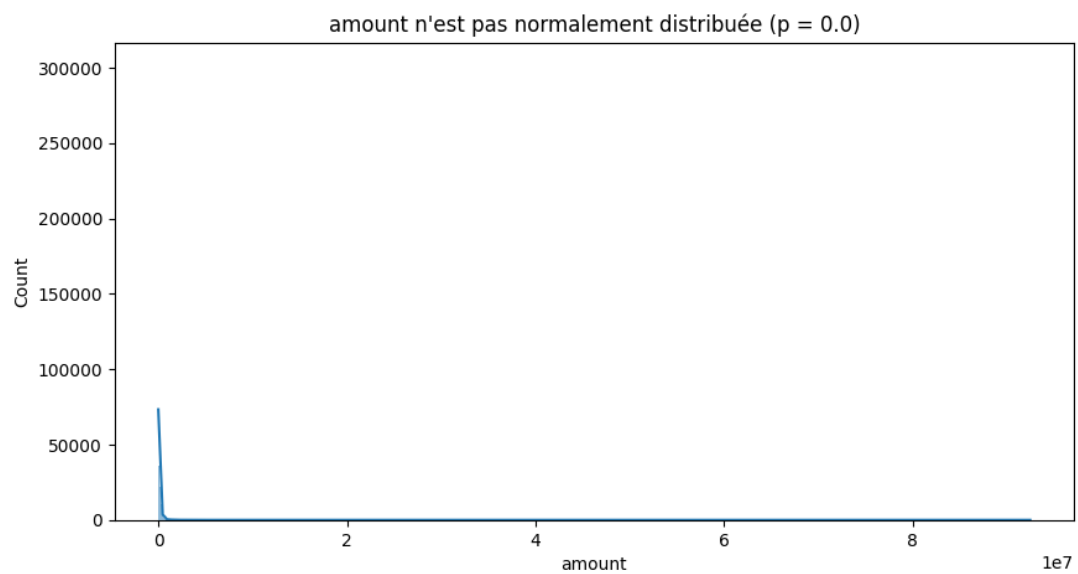
Analyse de la dispersion (boxplots) :

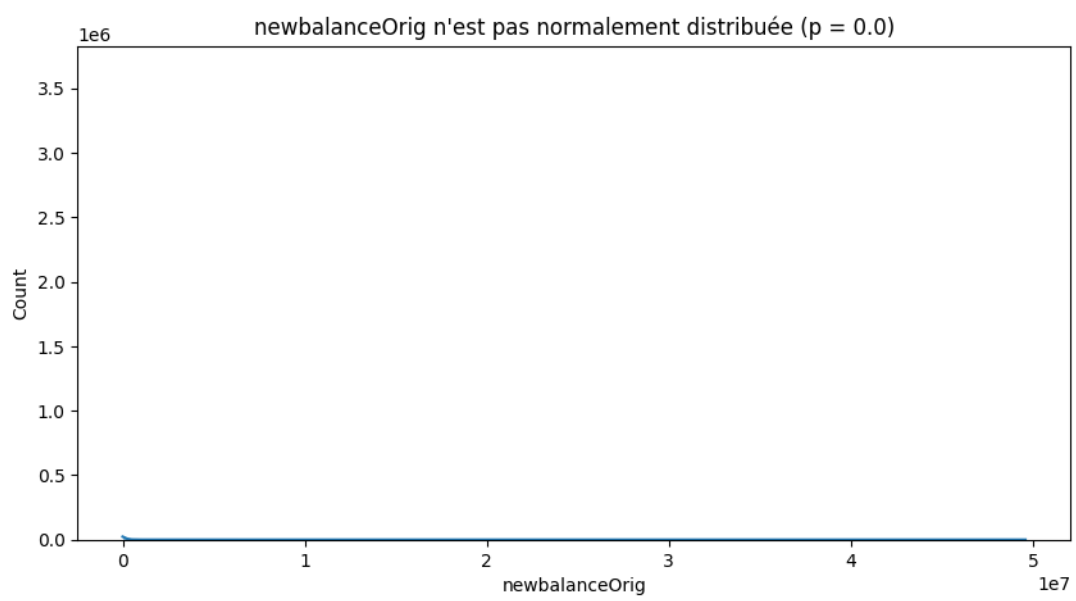
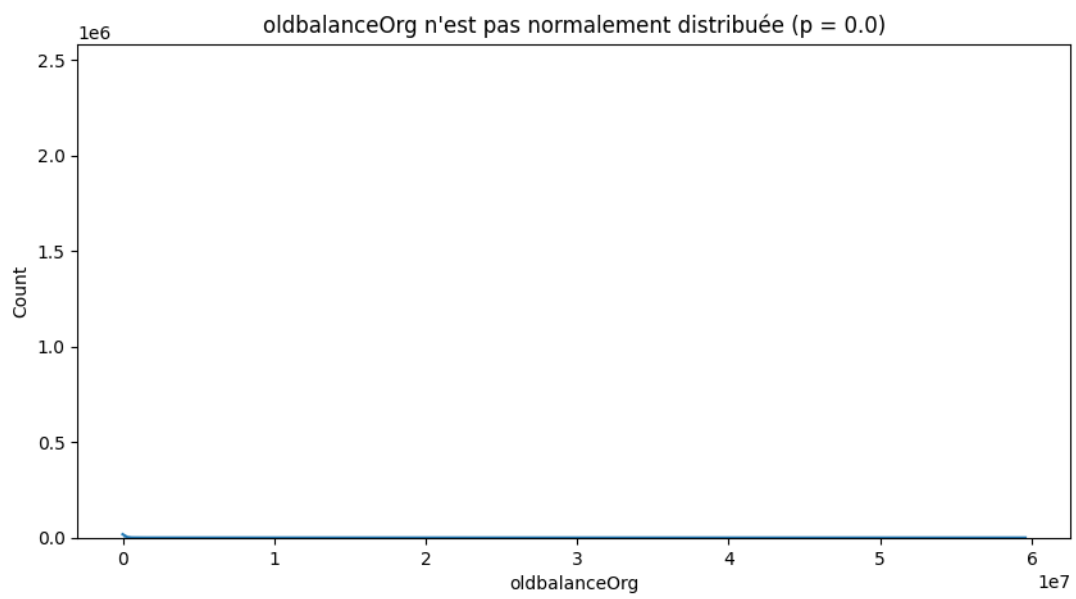


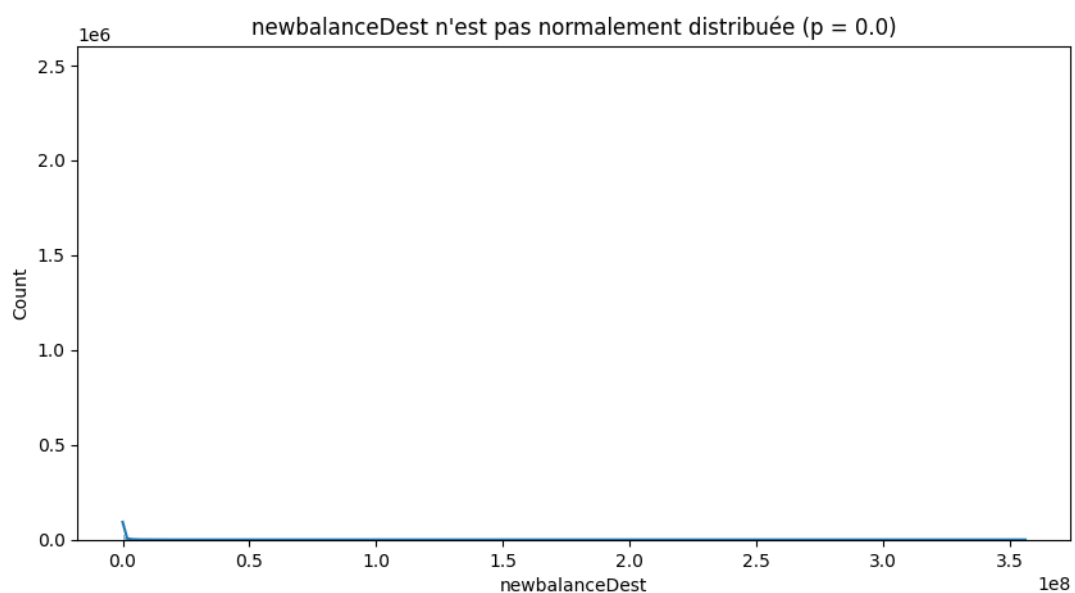
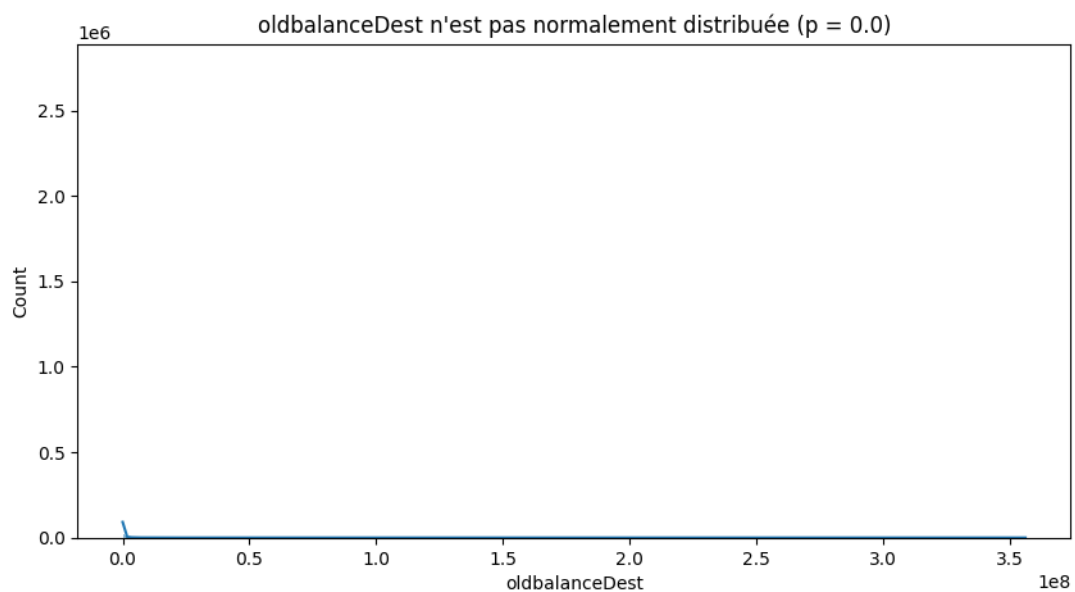




Analyse de la normalité (histogrammes + Shapiro) :



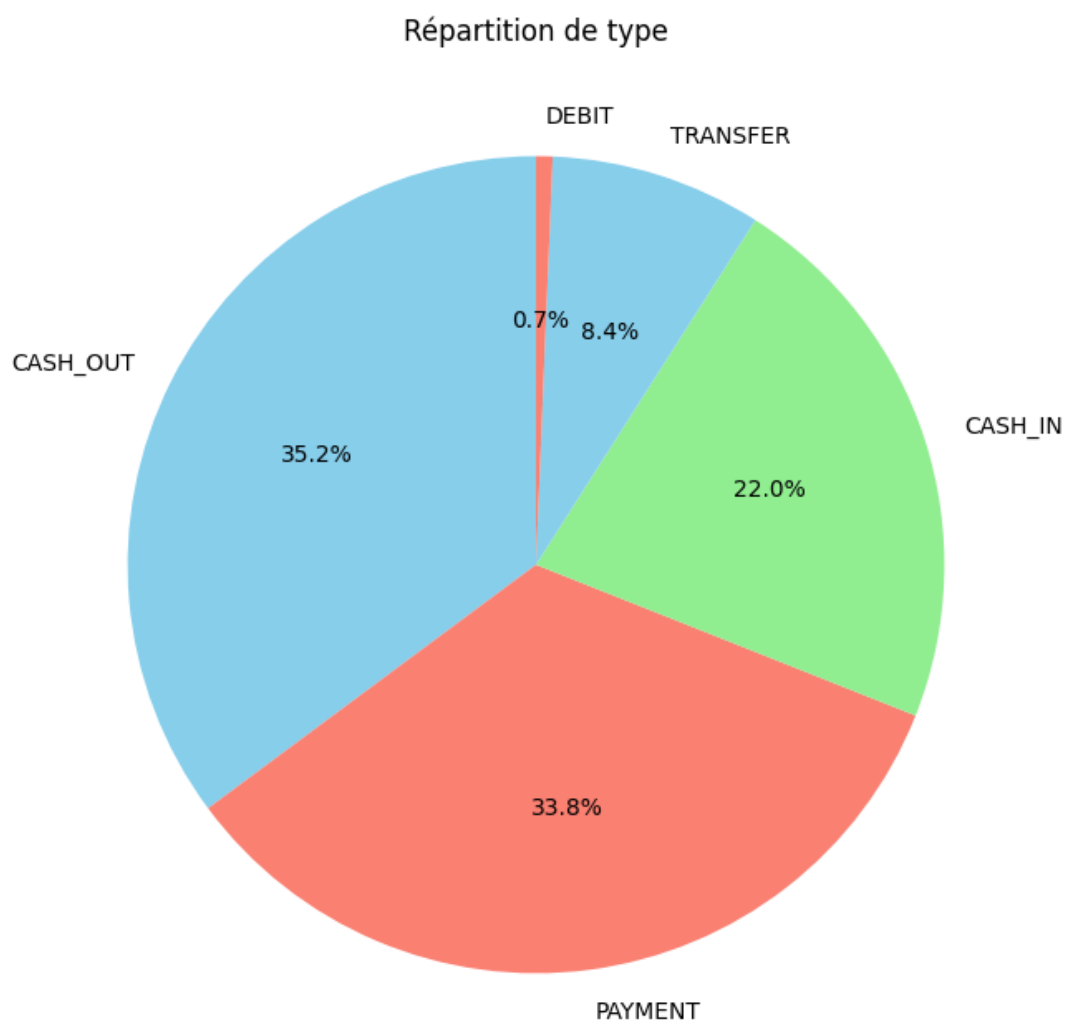




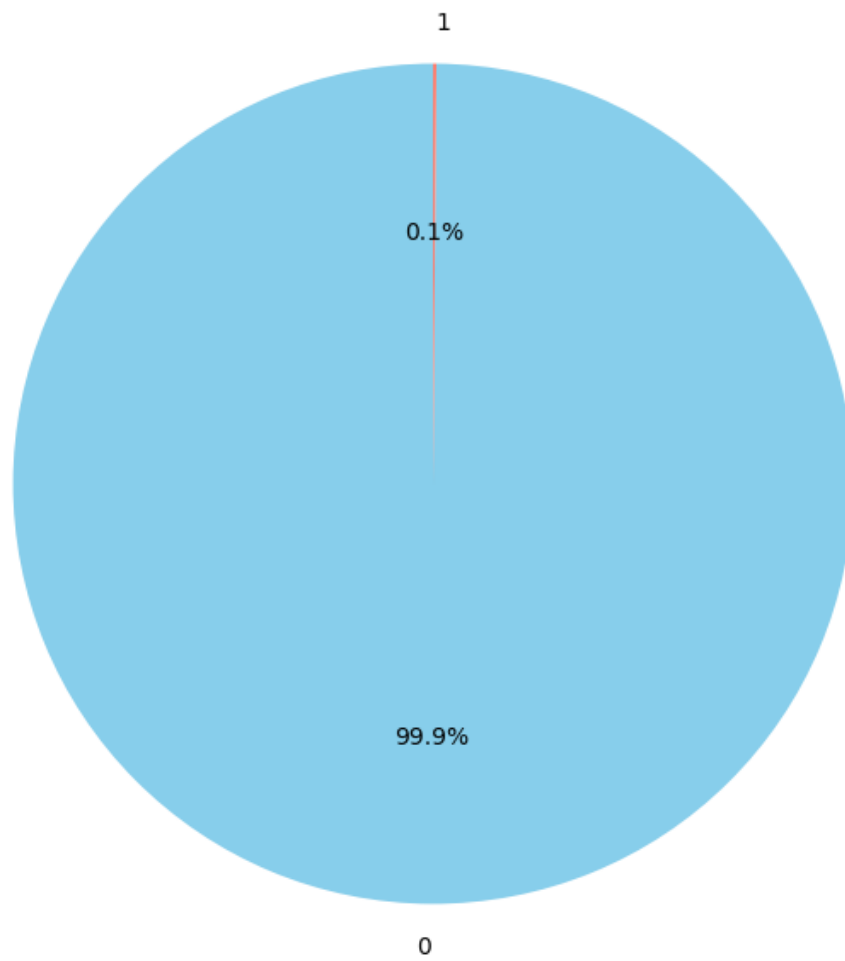
Statistique descriptive des variables qualitatives :

	type	isFraud	isFlaggedFraud
count	6362620	6.362620e+06	6.362620e+06
unique	5	NaN	NaN
top	CASH_OUT	NaN	NaN
freq	2237500	NaN	NaN
mean	NaN	1.290820e-03	2.514687e-06
std	NaN	3.590480e-02	1.585775e-03
min	NaN	0.000000e+00	0.000000e+00
25%	NaN	0.000000e+00	0.000000e+00
50%	NaN	0.000000e+00	0.000000e+00
75%	NaN	0.000000e+00	0.000000e+00

max NaN 1.000000e+00 1.000000e+00

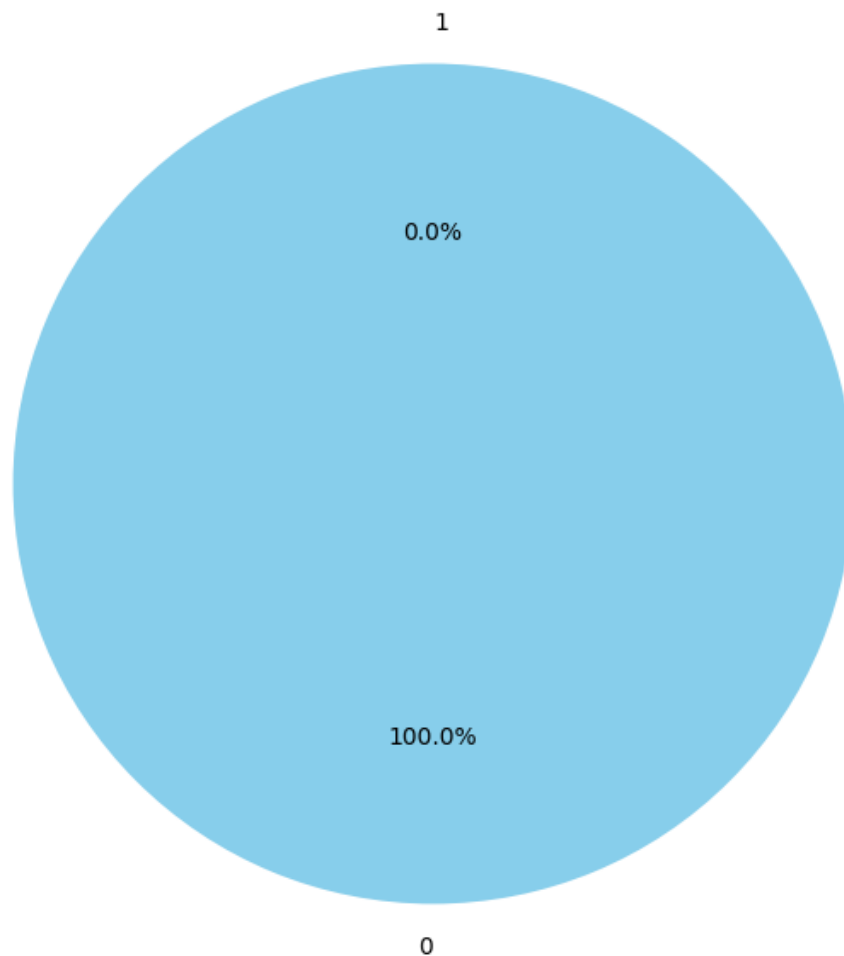


Répartition de isFraud



où 0 signifie pas de fraude et 1 signifie l'existence de fraude.

Répartition de isFlaggedFraud



Interprétation de l'analyse univariée

Variables Quantitatives

les caractéristiques quantitatives telles que (amount, oldbalanceOrg etc.) montrent une distribution fortement asymétrique avec des valeurs extrêmes (visibles dans les boxplots étirés et les histogrammes non-normaux). Cela suggère la nécessité de transformations (log, scaling) pour la modélisation. Par ailleurs, les **tests Shapiro-Wilks ($p=0.0$)** rejettent l'hypothèse de normalité pour toutes les variables, justifiant l'utilisation de méthodes robustes comme XGBoost.

Variables Qualitatives

L'analyse sur les types de transactions (représentés par la variable 'type') montre que les **CASH_OUT (35.2%)** et les **PAYMENT (22.0%)** dominent. Les fraudes sont souvent liées à des types spécifiques (à vérifier en bivariable). Nous pouvons également repérer un déséquilibre très important des classes de la variable cible 'isFraud' avec seulement **0.1% de fraude**, ce qui nécessite des techniques comme le rééchantillonnage ou des métriques focalisées sur la classe

minoritaire (recall, F1-score).

2. Analyse Bivariée

```
[6]: class BivariateAnalysis:

    def __init__(self, df):
        self.df = df
        self.variables = df.columns.tolist()
        self.quant_vars = ['amount', 'oldbalanceOrg', 'newbalanceOrig',
→'oldbalanceDest', 'newbalanceDest']
        self.qual_vars = ['type', 'isFraud']

    def BivariateQuant(self, variables):
        if not isinstance(variables, list) or len(variables) < 2:
            raise ValueError("'variables' doit être une liste d'au moins 2_
→variables")

        valid_vars = [var for var in variables if var in self.df.columns and
→pd.api.types.is_numeric_dtype(self.df[var])]

        if len(valid_vars) < 2:
            raise ValueError("Moins de 2 variables numériques valides_
→trouvées dans le DataFrame")

        df_clean = self.df[valid_vars].dropna()

        for i in range(len(valid_vars)):
            for j in range(i + 1, len(valid_vars)):
                x_var, y_var = valid_vars[i], valid_vars[j]

                plt.figure(figsize=(8, 6))
                sns.regplot(data=df_clean, x=x_var, y=y_var,
                            scatter_kws={'alpha': 0.5},
                            line_kws={'color': 'red'})
                plt.title(f"Relation entre {x_var} et {y_var}", pad=20)
                plt.tight_layout()
                plt.show()

                corr, pvalue = stats.spearmanr(df_clean[x_var],
→df_clean[y_var])
                significance = "significative" if pvalue < 0.05 else "non_
→significative"
                print(f"[{x_var} vs {y_var}] Corrélation {significance} (rho_
→= {corr:.2f}, p = {pvalue:.4f})")
                print("-" * 50)

        corr_matrix = df_clean.corr(method='spearman')
        mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

        plt.figure(figsize=(10, 8))
```

```

sns.heatmap(corr_matrix, mask=mask, annot=True, fmt=".2f",
→ cmap="coolwarm", square=True)
plt.title("Matrice de corrélation (Spearman)", pad=20)
plt.tight_layout()
plt.show()

def BivariateQual(self):
    for i in self.qual_vars:
        for j in self.qual_vars:
            if i != j:
                contingency_table = pd.crosstab(self.df[i], self.df[j])
                print(f"\n## Test du Chi2 entre {i} et {j} ##")
                chi2, pvalue, dof, expected = stats.
→ chi2_contingency(contingency_table)
                if pvalue < 0.05:
                    print(f" Association entre {i} et {j}, p = {pvalue:.
→ 4f}")
                else:
                    print(f" Aucune association entre {i} et {j}, p =
→ {pvalue:.4f}")

    def BivariateQuantQual(self):
        quant_vars = [var for var in self.quant_vars if var in self.df.
→ columns and pd.api.types.is_numeric_dtype(self.df[var])]
        qual_vars = [var for var in self.qual_vars if var in self.df.columns
→ and not pd.api.types.is_numeric_dtype(self.df[var])]

        if not quant_vars or not qual_vars:
            raise ValueError("Au moins une variable quantitative et une
→ qualitative sont nécessaires")

        df_clean = self.df[quant_vars + qual_vars].dropna()

        for quant_var in quant_vars:
            for qual_var in qual_vars:
                try:
                    groups = [group[quant_var].values for name, group in
→ df_clean.groupby(qual_var)]
                    h_stat, pvalue = stats.kruskal(*groups)
                    significance = "significative" if pvalue < 0.05 else "non
→ significative"

                    print(f"\nTest Kruskal-Wallis entre {quant_var} et
→ {qual_var} :")
                    print(f"H = {h_stat:.2f}, p = {pvalue:.4f} → Association
→ {significance}")

                plt.figure(figsize=(10, 6))
                sns.boxplot(x=qual_var, y=quant_var, data=df_clean)
                sns.stripplot(x=qual_var, y=quant_var, data=df_clean,

```



```

        color='black', alpha=0.3, jitter=True)
plt.title(f"{quant_var} par {qual_var} (p = {pvalue:.
→4f})", pad=20)

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

except ValueError as e:
    print(f"\nErreur avec {quant_var} et {qual_var} : {e}")

def lancer_analyse(self):
    print("\n--- Analyse bivariée quantitative ---")
    self.BivariateQuant(self.quant_vars)

    print("\n--- Analyse bivariée qualitative ---")
    self.BivariateQual()

    print("\n--- Analyse croisée quanti vs quali ---")
    self.BivariateQuantQual()

    print("\nFIN DE L'ANALYSE BIVARIÉE")

```

```

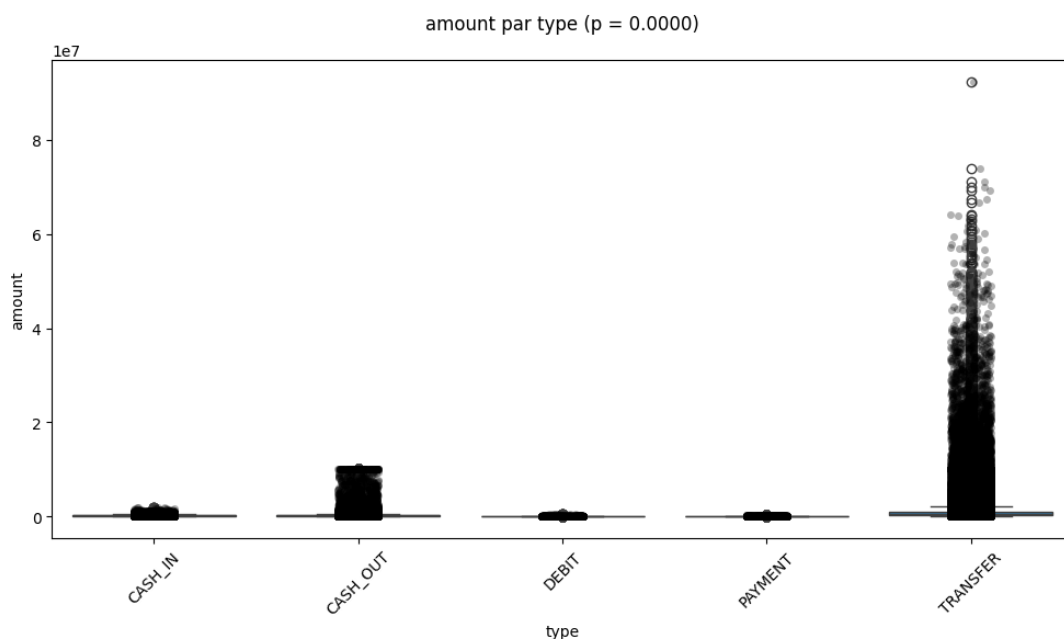
[23]: if __name__ == "__main__":
        # Exemple de test rapide

        analyse = BivariateAnalysis(data[['amount', 'oldbalanceOrig', '
→newbalanceOrig', 'oldbalanceDest', 'newbalanceDest', 'type', 'isFraud']])
        analyse.BivariateQuantQual()

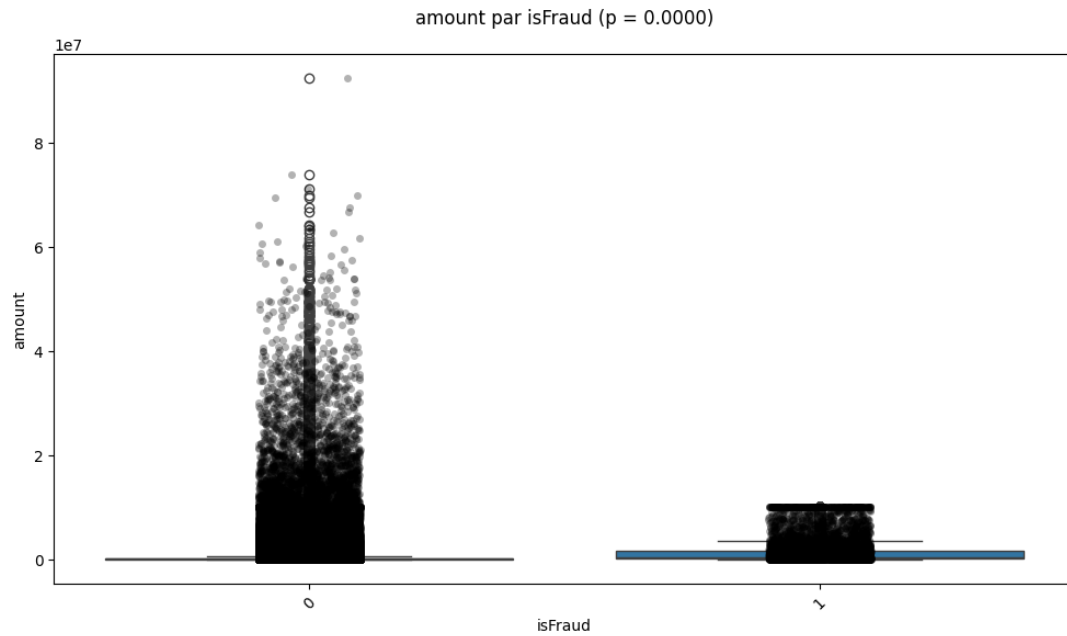
```

Test Kruskal-Wallis entre amount et type :

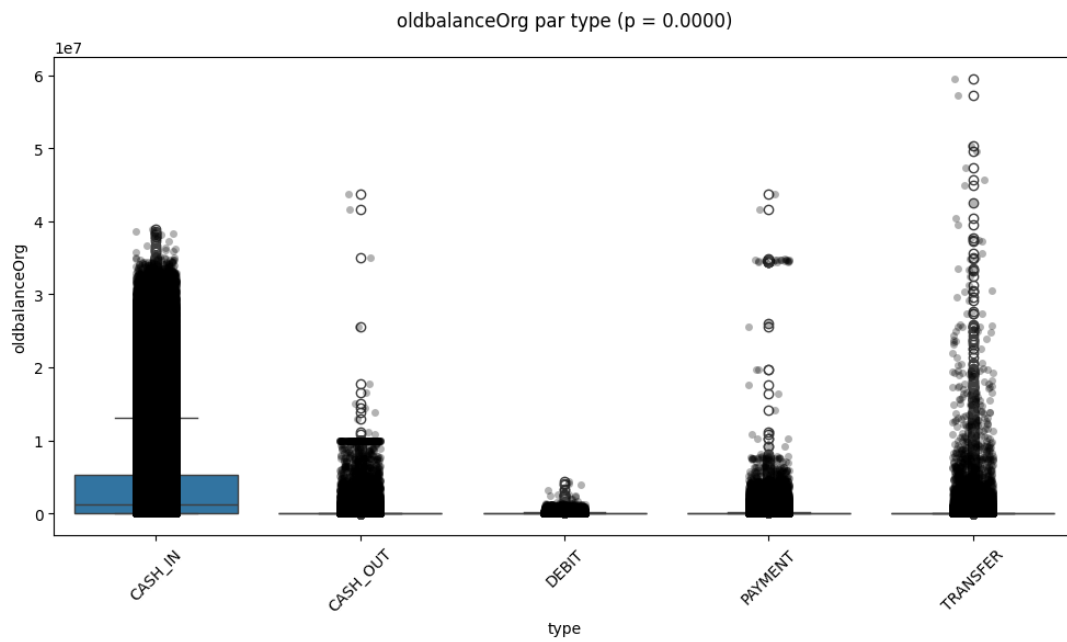
$H = 3906986.68$, $p = 0.0000$ → Association significative



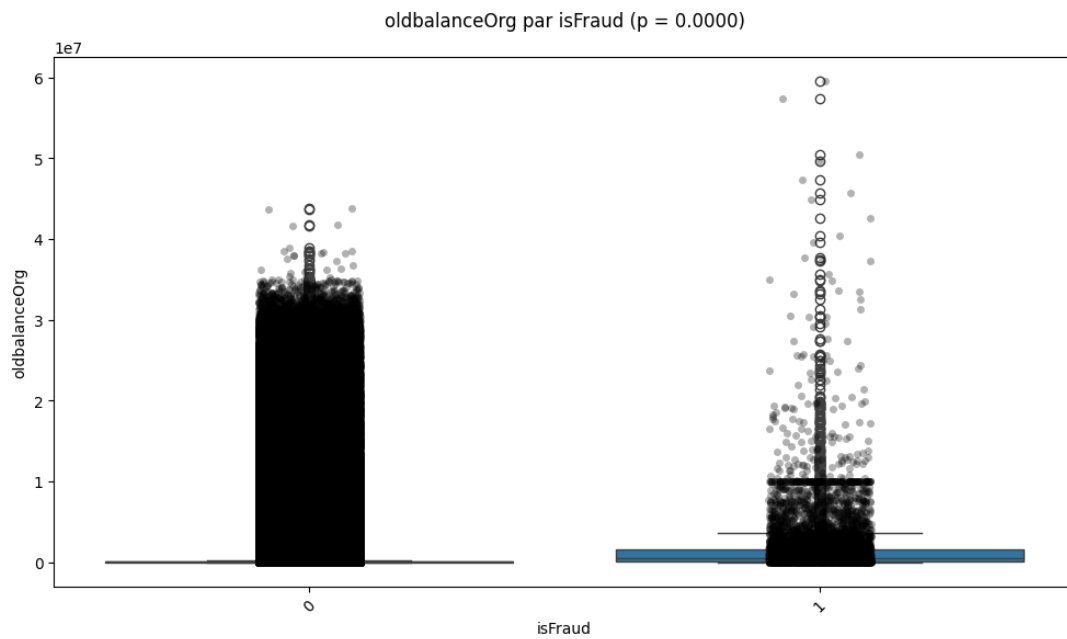
Test Kruskal-Wallis entre amount et isFraud :
 $H = 8273.37$, $p = 0.0000 \rightarrow$ Association significative



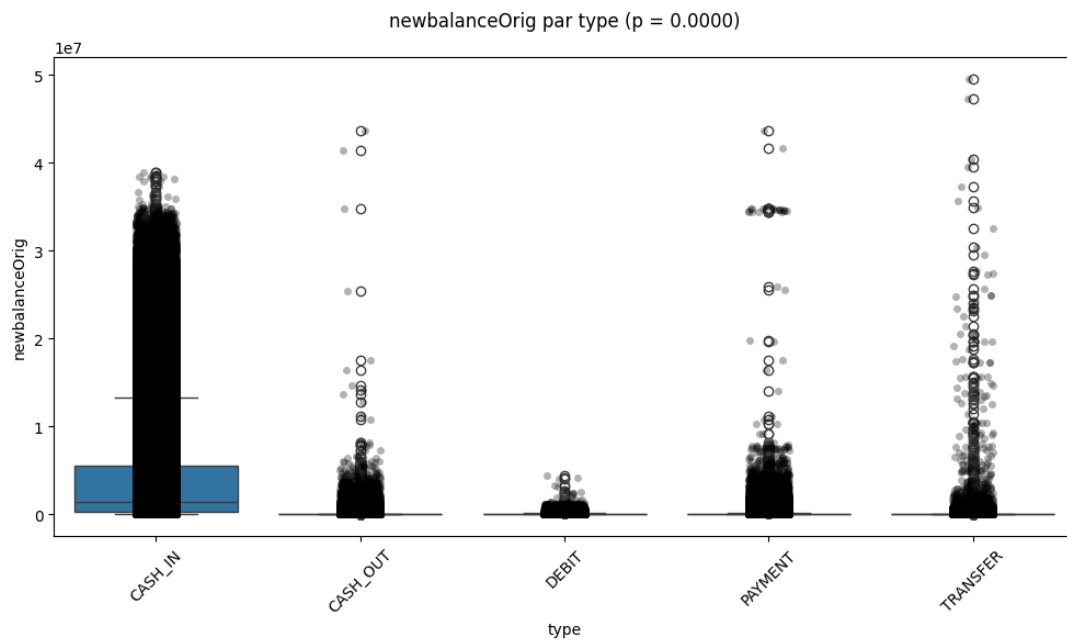
Test Kruskal-Wallis entre oldbalanceOrg et type :
 $H = 1868488.64$, $p = 0.0000 \rightarrow$ Association significative



Test Kruskal-Wallis entre oldbalanceOrg et isFraud :
 $H = 9892.17$, $p = 0.0000 \rightarrow$ Association significative

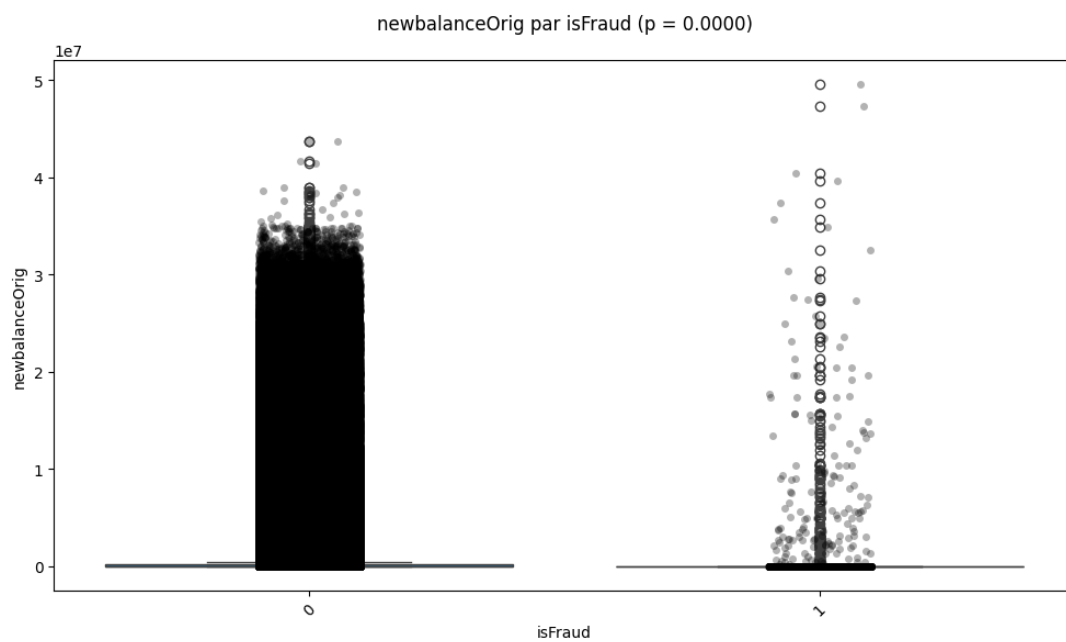


Test Kruskal-Wallis entre newbalanceOrig et type :
 $H = 4038804.73$, $p = 0.0000 \rightarrow$ Association significative



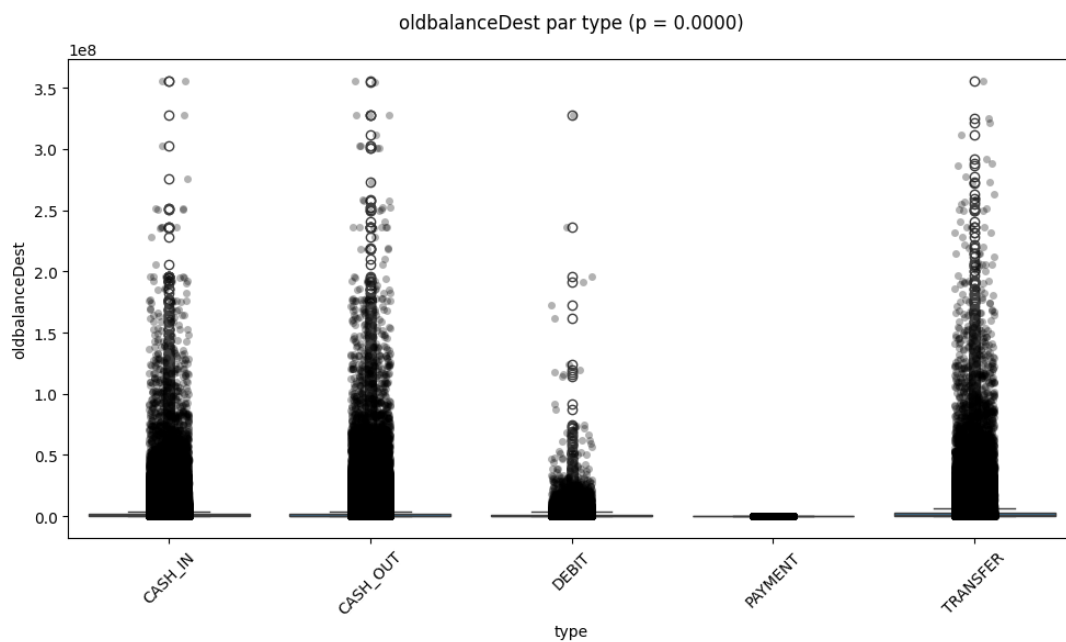
Test Kruskal-Wallis entre newbalanceOrig et isFraud :

$H = 4999.38$, $p = 0.0000 \rightarrow$ Association significative



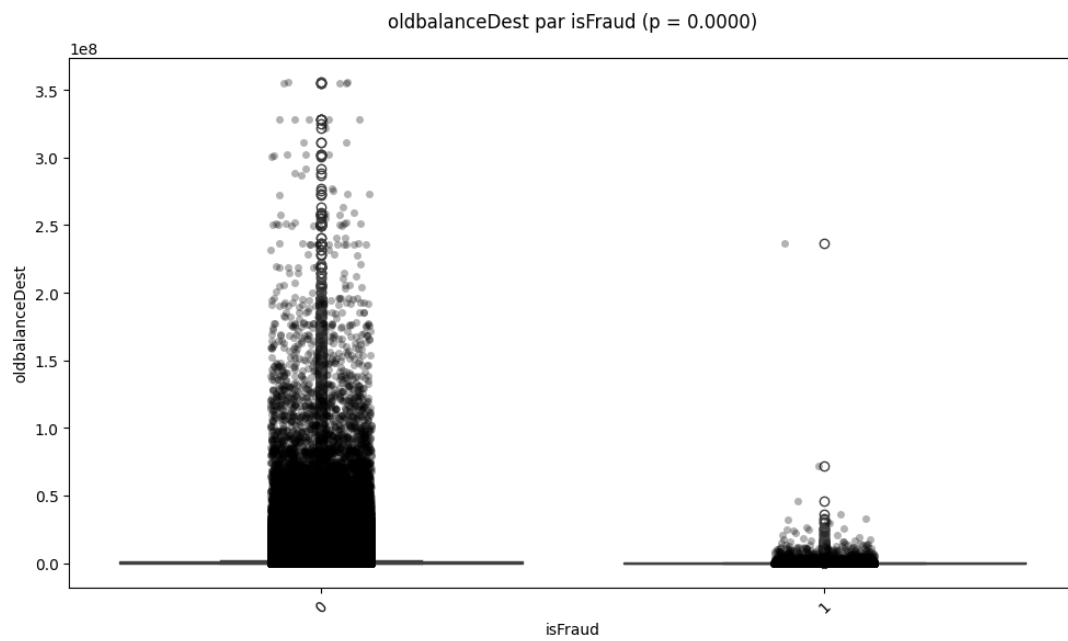
Test Kruskal-Wallis entre oldbalanceDest et type :

$H = 3517554.62$, $p = 0.0000 \rightarrow$ Association significative

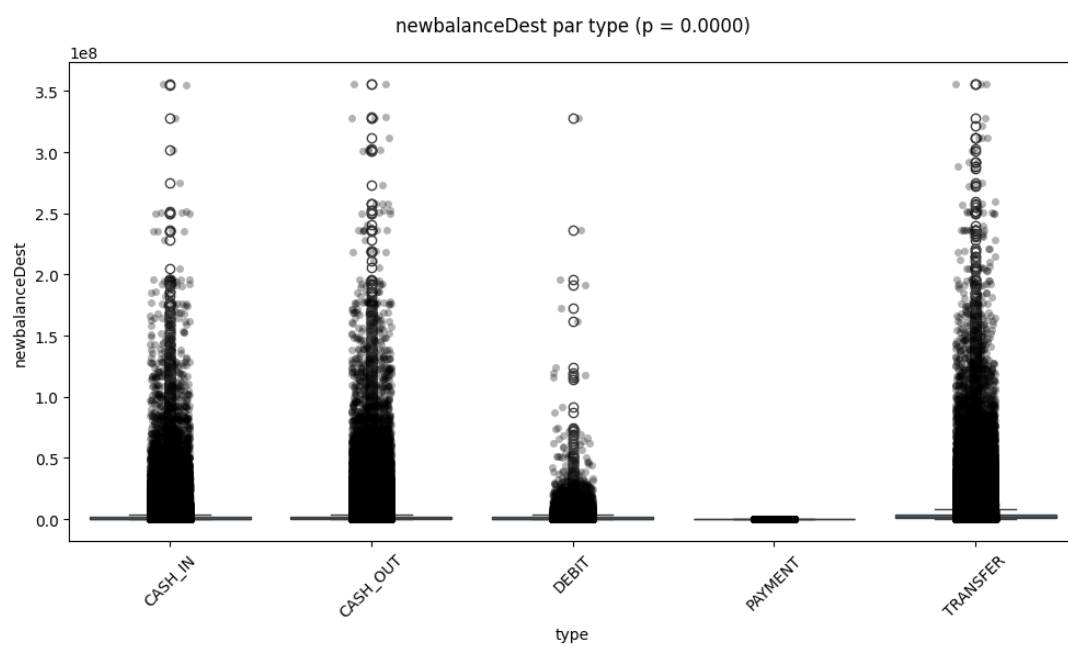


Test Kruskal-Wallis entre oldbalanceDest et isFraud :

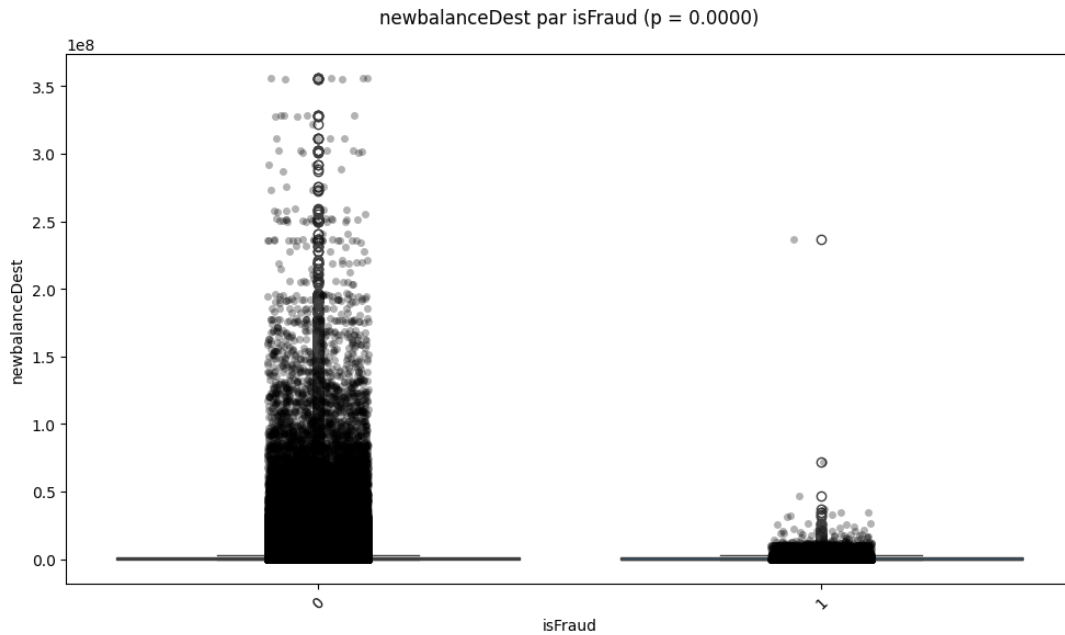
$H = 1869.46$, $p = 0.0000 \rightarrow$ Association significative



Test Kruskal-Wallis entre newbalanceDest et type :
 $H = 4170791.50$, $p = 0.0000 \rightarrow$ Association significative



Test Kruskal-Wallis entre newbalanceDest et isFraud :
 $H = 170.84$, $p = 0.0000 \rightarrow$ Association significative



Interprétation de l'analyse univariée

(NB : Compte tenu de la taille de la base de donnée et de la performance des outils utilisés pour ce projet, certains résultats de cette section ne sont pas affichés)

Variables

Matrice de Spearman: Aucune corrélation forte (>0.5) entre les variables quantitatives, sauf entre `oldbalanceOrig/newbalanceOrig` (0.99) et `oldbalanceDest/newbalanceDest` (0.97), suggérant des redondances à traiter (PCA ou suppression).

Relations non linéaires: Les regplots montrent des nuages de points dispersés, indiquant que des modèles linéaires seraient peu adaptés → choix de XGBoost justifié par sa capacité à capturer des relations complexes.

Fraude vs Montant: Les transactions frauduleuses ont des montants médians plus élevés (boxplot `amount` par `isFraud`), mais avec une grande variance. XGBoost peut gérer cette hétérogénéité via ses splits optimaux.

Type de transaction: Les `TRANSFER` et `CASH_OUT` montrent des distributions de soldes différentes pour les fraudes ($p=0.0000$), confirmant leur importance comme features.

Association `type/isFraud`: Lien significatif ($p=0.0000$), validant l'inclusion de `type` dans le modèle.

4.1. Analyse Multivariée avant le Feature Engineering

- Encodage
- Normalisation
- Définition du target et des features
- Instanciation de PCA

```
[7]: # Suppression des variables inutiles
data_1 = data.copy()
data.drop(columns=["isFlaggedFraud", "nameOrig", "nameDest"], inplace=True)

# Conversion des variables en catégories
var_categorielle = ["isFraud", "type"]
for var in var_categorielle:
    data[var] = data[var].astype("category")

# Encoding des variables catégorielles
df_dummies_0 = pd.get_dummies(data[["type"]], prefix=["type"],
    → prefix_sep="_", dtype=int)
data_0 = pd.concat([data, df_dummies_0], axis=1)
data_0.drop(columns=["type"], inplace=True)

# Normalisation des variables
data_0[[
    'amount', 'oldbalanceOrg', 'newbalanceOrig',
    'oldbalanceDest', 'newbalanceDest']] = (data_0[['amount', 'oldbalanceOrg',
    → 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest']] -
    → data_0[['amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest',
    → 'newbalanceDest']].mean())/data_0[['amount', 'oldbalanceOrg',
    → 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest']].std()
data_vf = data_0.copy()
```

```
[8]: class MultivariateAnalyser:
    def __init__(self, df):
        self.data = df

    def pca(self):

        # Définition des features et de la variables cibles
        features = self.data.drop(columns = "isFraud")
        target = self.data["isFraud"]
        pca = PCA()
        features_pca = pca.fit_transform(features)

        # Détermination des composants principales
        composant_principale = pd.DataFrame(
            {
                "Dimension" : ["PCA_" + str(x) for x in range(features.
    → shape[1])],
                "Valeur propre" : np.round(pca.explained_variance_,3),
                "Taux_variance" : np.round(pca.explained_variance_ratio_,3),
                "Taux_variance_cum" : np.round(np.cumsum(pca.
    → explained_variance_ratio_),3)
            }
        )
```

```

# Impact de variables initiales sur les composantes
vecteur_propre = pd.DataFrame(
    pca.components_.T,
    columns=["PCA_" + str(x+1) for x in range(features.shape[1])],
    index = features.columns.to_list()
)

# Tri des variables par leur contribution absolue maximale à une
→composante principale
vecteur_propre_sorted = vecteur_propre.abs().max(axis=1).
→sort_values(ascending=False).index
vecteur_propre = vecteur_propre.loc[vecteur_propre_sorted]

# Affichage du tableau croisé des 5 premières variables les plus
→contributives
vecteur_propre_5 = vecteur_propre.sort_values(by="PCA_1",
→ascending=False)

# Représentation graphique de l'ACP

# Créer le cercle de corrélation
coeff = np.transpose(pca.components_[0:2, :])
n = coeff.shape[0]
xs = np.array([1, 0])
ys = np.array([0, 1])
feature_names = features.columns.to_list()
# Créer la figure
plt.figure(figsize=(10, 10))

# Placer les vecteurs des variables
for i in range(n):
    plt.arrow(0, 0, coeff[i, 0], coeff[i, 1], color='k', alpha=0.9,
→head_width=0.02)
    plt.text(coeff[i, 0] * 1.15, coeff[i, 1] * 1.15,
→feature_names[i], color='k', ha='center', va='center')

# Placer le cercle unitaire
circle = plt.Circle((0, 0), 1, color='gray', fill=False,
→linestyle='--')
plt.gca().add_artist(circle)

# Ajuster les limites et les axes
plt.xlim(-1, 1)
plt.ylim(-1, 1)
plt.axhline(0, color='gray', linewidth=1)
plt.axvline(0, color='gray', linewidth=1)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('Cercle de corrélation ACP des fraudes')

```



```

    # Afficher la figure
    plt.show()

    return composant_principale, vecteur_propre_5

def lancer_analyse(self):

    """
    Cela permet de lancer l'analyse
    """
    print("\n --Analyse Multivariée -- \n")
    cp, vp5 = self.pca()
    print("-- Les Composants Principaux --")
    print(f"\n {cp} \n")

    print("-- Influence de chaque variable dans les composants --")
    print(f"{vp5} \n")
    print("\n FIN DE L'ANALYSE BIVARIÉE")

```

```

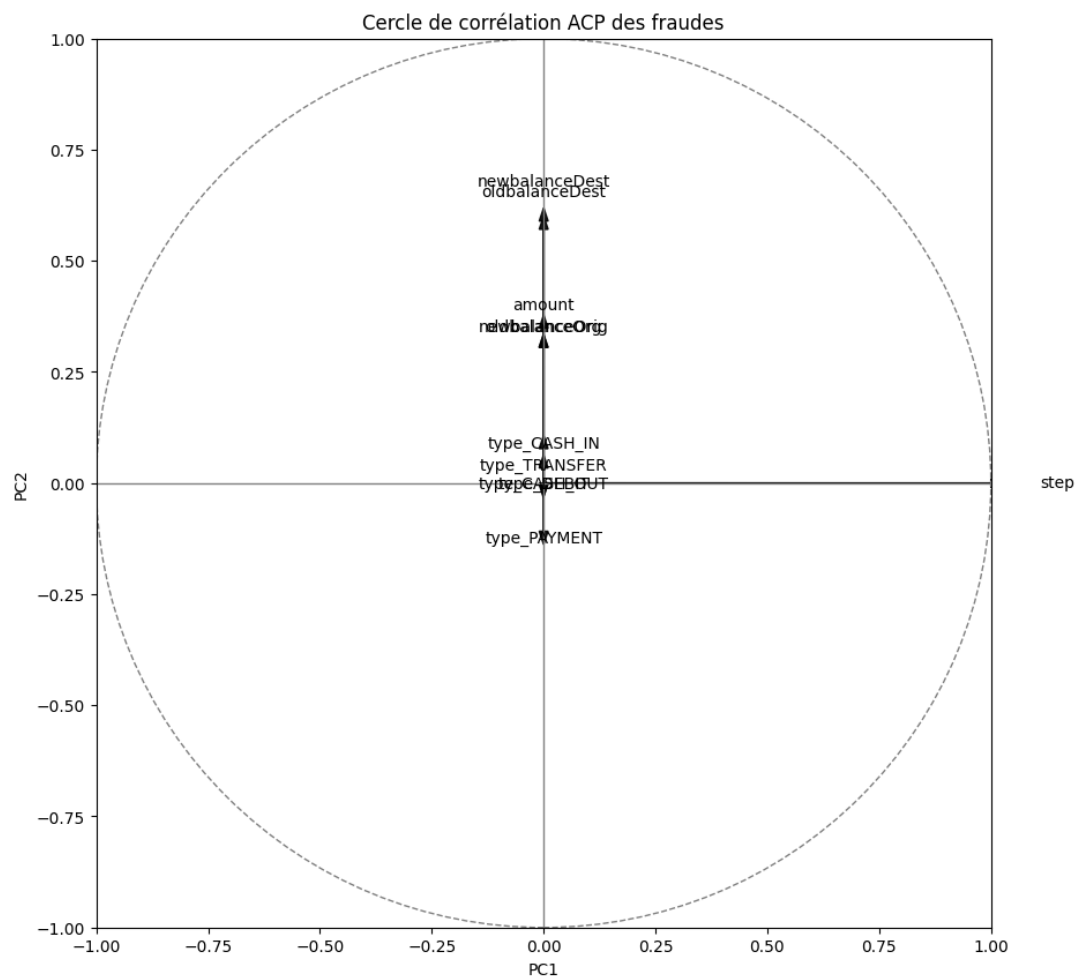
[10]: an_before = MultivariateAnalyser(data_vf)
      an_before.lancer_analyse()

```

```

--Analyse Multivariée --

```



-- Les Composants Principaux --

	Dimension	Valeur propre	Taux_variance	Taux_variance_cum
0	PCA_0	20258.392	1.0	1.0
1	PCA_1	2.285	0.0	1.0
2	PCA_2	2.020	0.0	1.0
3	PCA_3	0.787	0.0	1.0
4	PCA_4	0.330	0.0	1.0
5	PCA_5	0.181	0.0	1.0
6	PCA_6	0.085	0.0	1.0
7	PCA_7	0.008	0.0	1.0
8	PCA_8	0.007	0.0	1.0
9	PCA_9	0.001	0.0	1.0
10	PCA_10	0.000	0.0	1.0

-- Influence de chaque variable dans les composants --

	PCA_1	PCA_2	PCA_3	PCA_4	PCA_5	PCA_6 \
step	1.000000	-0.000229	0.000221	-0.000029	0.000071	-0.000027
oldbalanceDest	0.000194	0.570943	-0.255725	-0.387041	-0.060993	0.022902
newbalanceDest	0.000182	0.589483	-0.290709	-0.193589	-0.058678	-0.051319
amount	0.000157	0.348623	-0.213127	0.893284	-0.018864	-0.044260

type_PAYMENT	0.000016	-0.108373	-0.017541	-0.035981	-0.672788	-0.466415
type_TRANSFER	0.000013	0.033807	-0.038587	0.095204	-0.039269	0.103640
type_CASH_IN	0.000012	0.077308	0.142813	-0.002966	-0.020406	0.755084
type_DEBIT	0.000002	-0.000387	-0.001019	-0.002575	-0.000706	0.007656
type_CASH_OUT	-0.000043	-0.002356	-0.085667	-0.053682	0.733169	-0.399966
oldbalanceOrg	-0.000071	0.305094	0.621940	0.030464	0.016009	-0.158605
newbalanceOrig	-0.000072	0.305032	0.623160	0.024146	0.009264	-0.106115

		PCA_7	PCA_8	PCA_9	PCA_10	PCA_11
step	0.000013	5.942024e-07	0.000020	0.000002	-4.759636e-18	
oldbalanceDest	-0.002979	-7.877240e-02	-0.669598	-0.002970	1.107113e-13	
newbalanceDest	-0.004979	8.303665e-02	0.719391	0.001221	-1.192488e-13	
amount	-0.119279	-1.067870e-02	-0.135705	0.003236	1.946448e-14	
type_PAYMENT	-0.263827	-2.162632e-01	0.007008	0.006571	4.472136e-01	
type_TRANSFER	0.847525	-2.391180e-01	0.019960	0.012131	4.472136e-01	
type_CASH_IN	-0.387563	-2.161942e-01	0.068103	-0.038309	4.472136e-01	
type_DEBIT	0.016236	8.882881e-01	-0.102732	0.007735	4.472136e-01	
type_CASH_OUT	-0.212370	-2.167128e-01	0.007661	0.011873	4.472136e-01	
oldbalanceOrg	0.045042	7.612686e-03	0.000142	-0.701200	4.983592e-13	
newbalanceOrig	0.015412	-4.525792e-03	0.000977	0.711645	-5.065791e-13	

FIN DE L'ANALYSE MULTIVARIEE

Interprétation de l'ACP avant le feature engineering

Les variables de solde (old/newbalanceOrg/Dest) contribuent fortement aux premières composantes, mais avec des redondances (vecteurs superposés). PCA_0 capture 100% de la variance (artéfact dû au scaling?), nécessitant une vérification des pré-traitements. La variable 'step', représentant l'heure de l'opération durant la collecte des données, explique quasi toute la variance → dominance inutile. Nous pouvons remarquer que les variables initiales apportent peu d'information sur le fait qu'une opération soit frauduleuse ou non.

3. Feature Engineering

- L'objectif principal ici c'est de créer des variables dérivées permettant de capter au moins les anomalies

```
[9]: # Variables dérivées
## | Vérifie si l'argent est vraiment retiré du compte client. Si 0 →
    → anomalie possible. |
data_1["DiffSoldeOrig"] = data_1["oldbalanceOrg"] - data_1["amount"] -
    → data_1["newbalanceOrig"]
data_1["DiffSoldeOrig"] = data_1["DiffSoldeOrig"].apply(lambda x: 1 if x != 0
    → else 0) # | Si 0 → anomalie possible. |

## | Pareil mais côté bénéficiaire |
data_1["DiffSoldeDest"] = data_1["oldbalanceDest"] + data_1["amount"] -
    → data_1["newbalanceDest"]
data_1["DiffSoldeDest"] = data_1["DiffSoldeDest"].apply(lambda x: 1 if x !=
    → 0 else 0) # | Si 0 → anomalie possible. |
```

```

## | Montant transféré par rapport au solde initial du client. Un % élevé est
→suspect. /
data_1["PourcentageMontantOrig"] = data_1["amount"] /
→(data_1["oldbalanceOrig"] + 1)

## | 1 si nameOrig == nameDest sinon 0 | Transaction entre le même compte :
→peut être une fraude déguisée. /
data_1["isSameAccount"] = data_1["nameOrig"] == data_1["nameDest"]
data_1["isSameAccount"] = data_1["isSameAccount"].apply(lambda x : 1 if x ==
→True else 0) # | 1 si nameOrig == nameDest sinon 0 | Transaction entre le
→même compte : peut être une fraude déguisée. /

## | activitéAnormale | 1 si (DiffSoldeOrig 0 OU DiffSoldeDest 0) sinon 0 /
→Comportement suspect basé sur les incohérences des soldes. /
data_1["activiteAnormale"] = data_1["DiffSoldeOrig"] + data_1["DiffSoldeDest"]
data_1["step_day"] = np.round(data_1["step"]/24).astype(int)

# Suppression des variables inutiles
data_1.drop(columns=["step", "isFlaggedFraud", "nameOrig", "nameDest"],
→inplace=True)

# Conversion des variables en catégories
var_categorielle = ["isFraud", "DiffSoldeOrig", "DiffSoldeDest",
→"isSameAccount", "activiteAnormale", "type"]
for var in var_categorielle:
    data_1[var] = data_1[var].astype("category")

```

Analyse Univariée & Bivariée après le feature engineering

```

[22]: # Resultats des analyses
if __name__ == "__main__":
    print("\nDÉBUT DE L'ANALYSE UNIVARIÉE\n")
    # Instanciation de la classe AnalyseUnivariee
    # et lancement de l'analyse univariée
    analyse_1 = AnalyseUnivariee(data_1[["DiffSoldeOrig", "DiffSoldeDest",
→"isSameAccount", "activiteAnormale"]])
    analyse_2 = BivariateAnalysis(data_1[["DiffSoldeOrig", "DiffSoldeDest",
→"isSameAccount", "activiteAnormale", "amount", "oldbalanceOrig",
→"newbalanceOrig", "oldbalanceDest", "newbalanceDest", "type", "isFraud"]])

    # Analyses des variables qualitatives
    analyse_1.analyser_quali()
    analyse_2.BivariateQual()
    analyse_2.BivariateQuantQual()

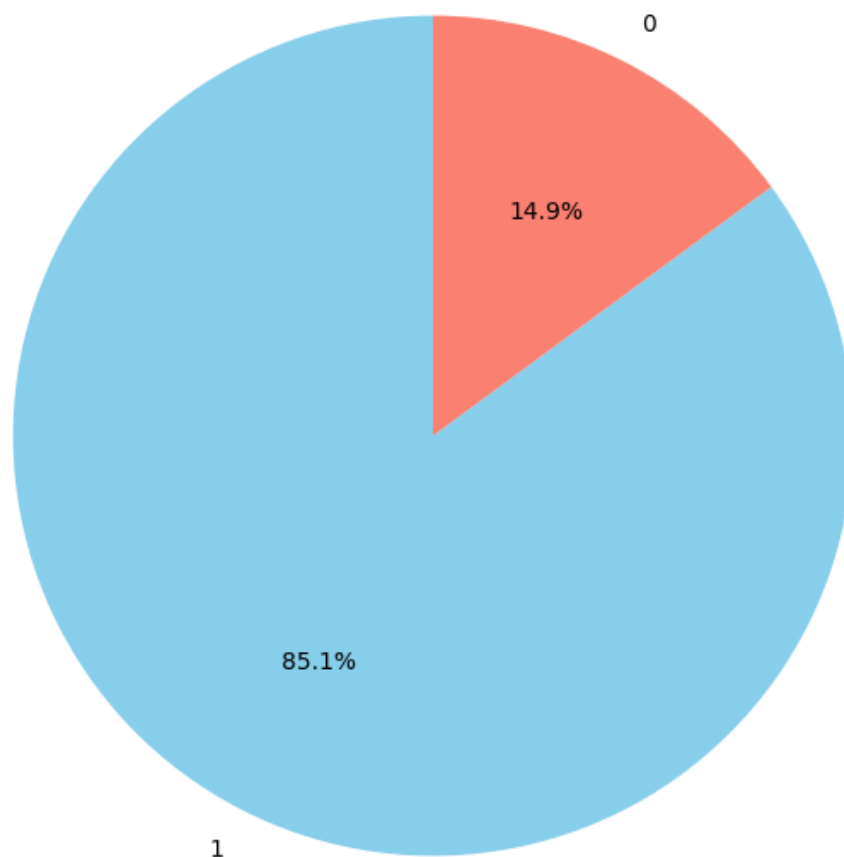
```

DÉBUT DE L'ANALYSE UNIVARIÉE

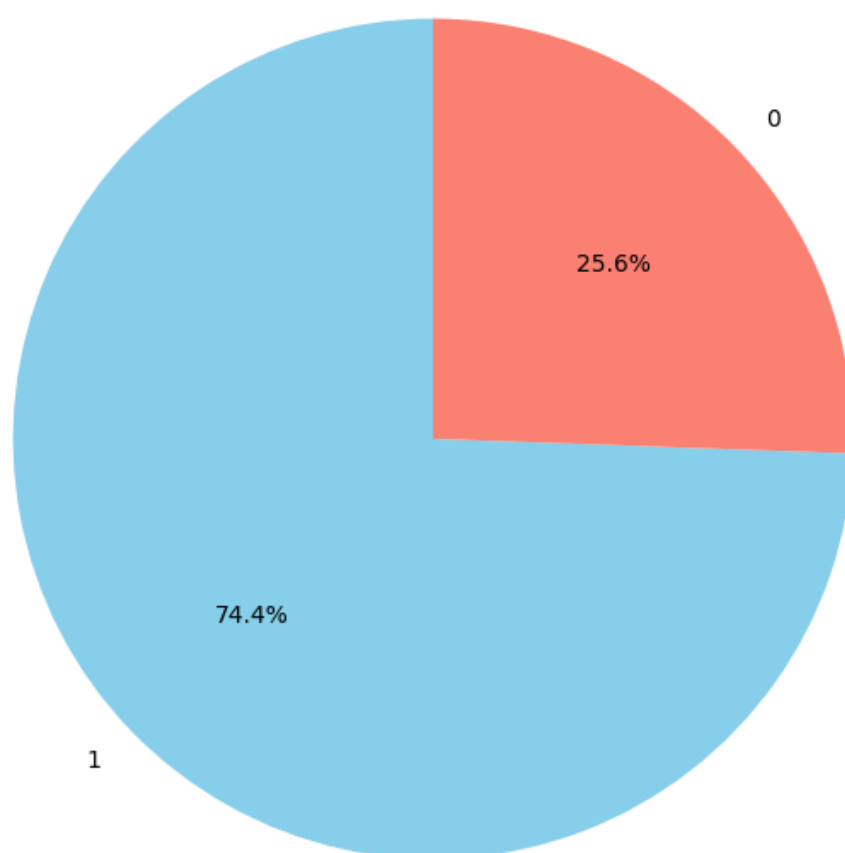
Statistique descriptive des variables qualitatives :

	DiffSoldeOrig	DiffSoldeDest	isSameAccount	activiteAnormale
count	6362620	6362620	6362620	6362620
unique	2	2	1	3
top	1	1	0	2
freq	5413997	4736674	6362620	3910108

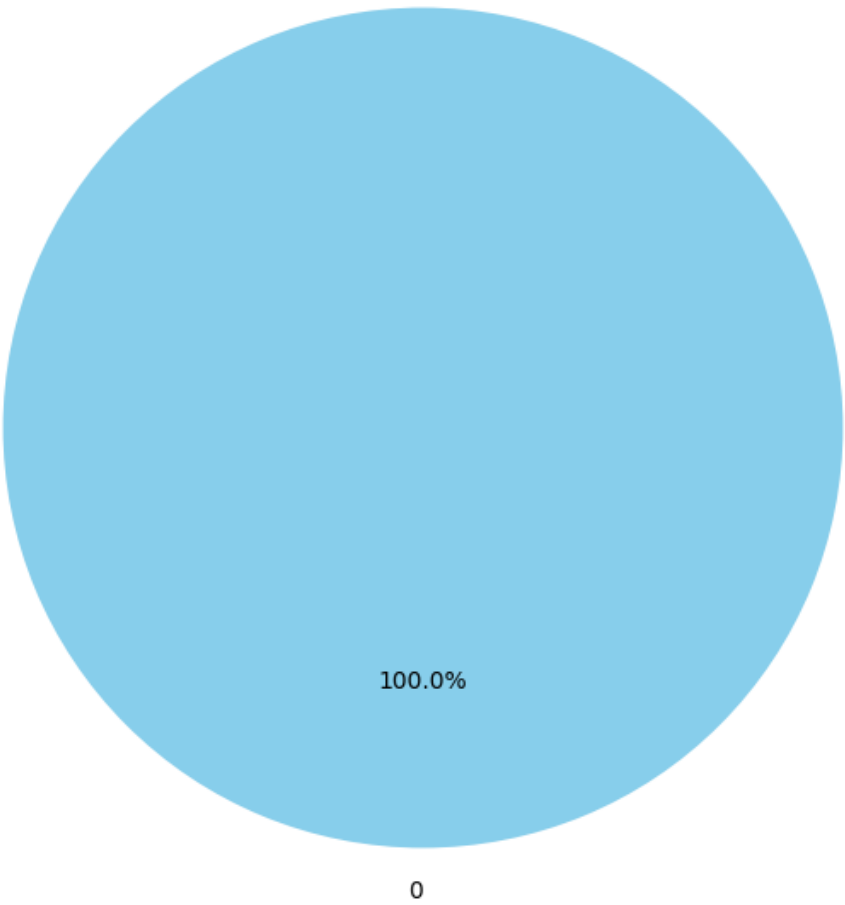
Répartition de DiffSoldeOrig



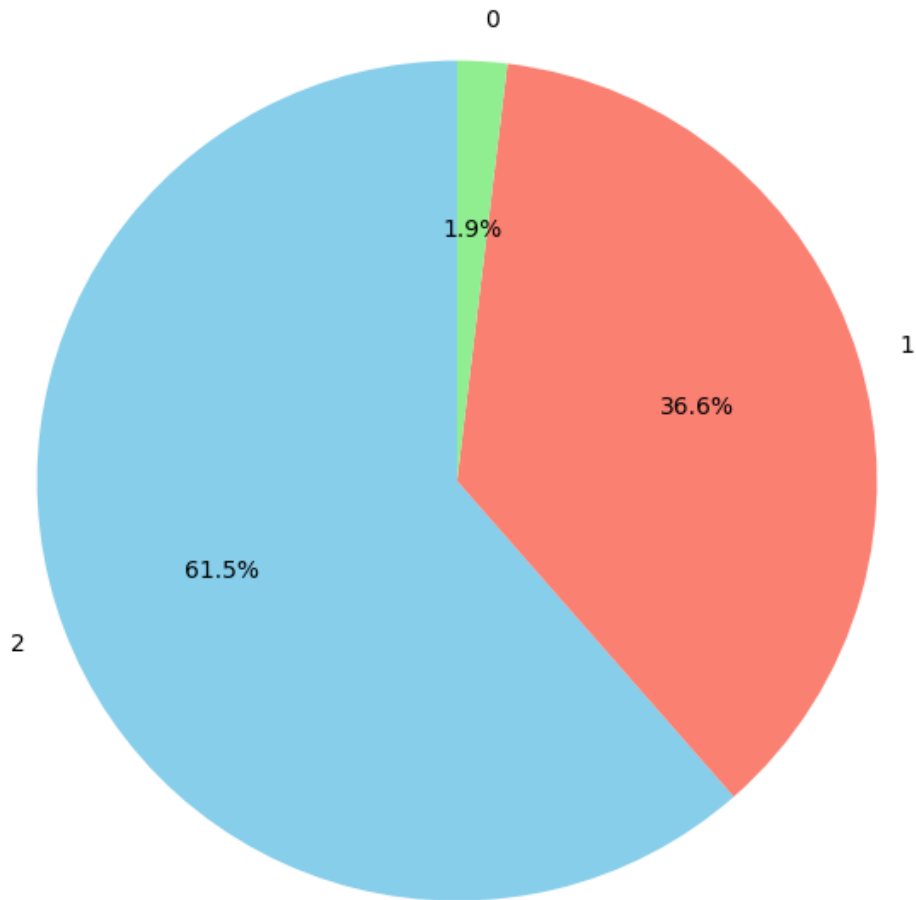
Répartition de DiffSoldeDest



Répartition de isSameAccount



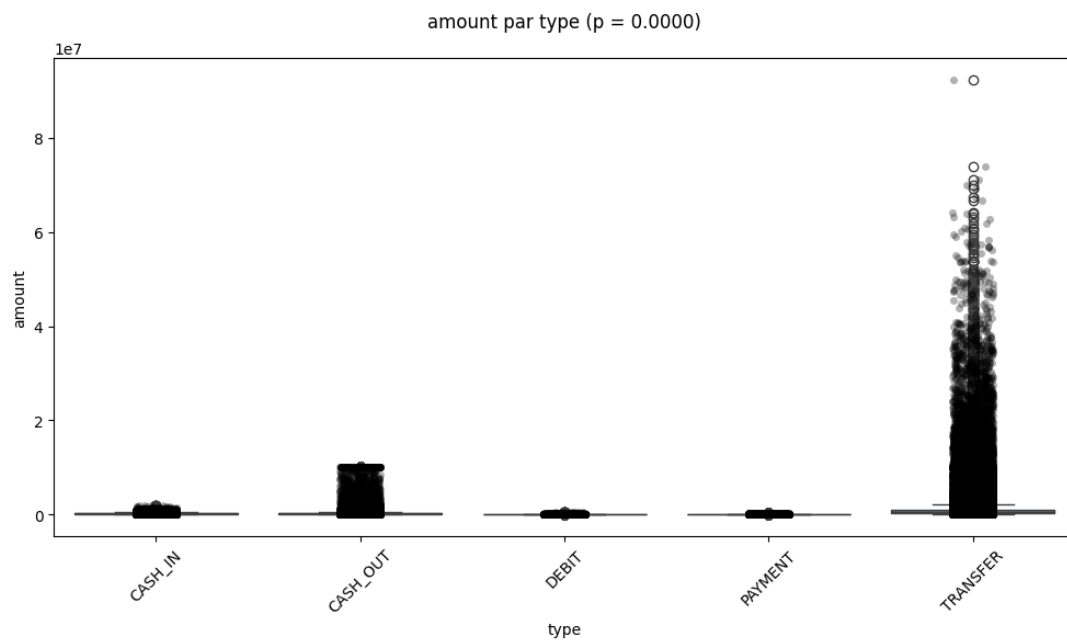
Répartition de activiteAnormale



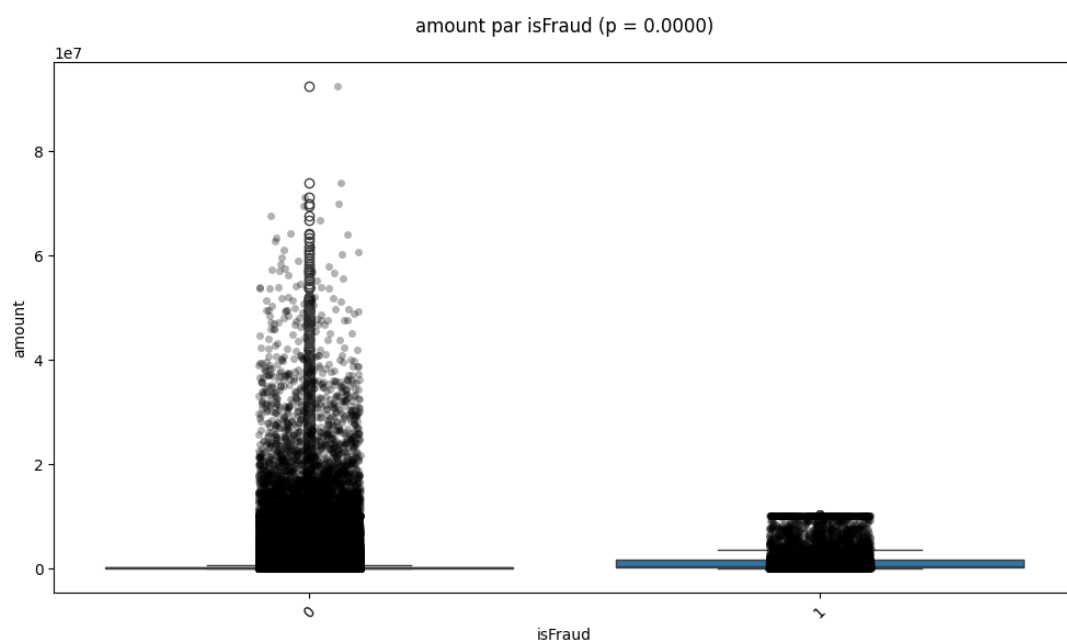
```
## Test du Chi2 entre type et isFraud ##  
Association entre type et isFraud, p = 0.0000
```

```
## Test du Chi2 entre isFraud et type ##  
Association entre isFraud et type, p = 0.0000
```

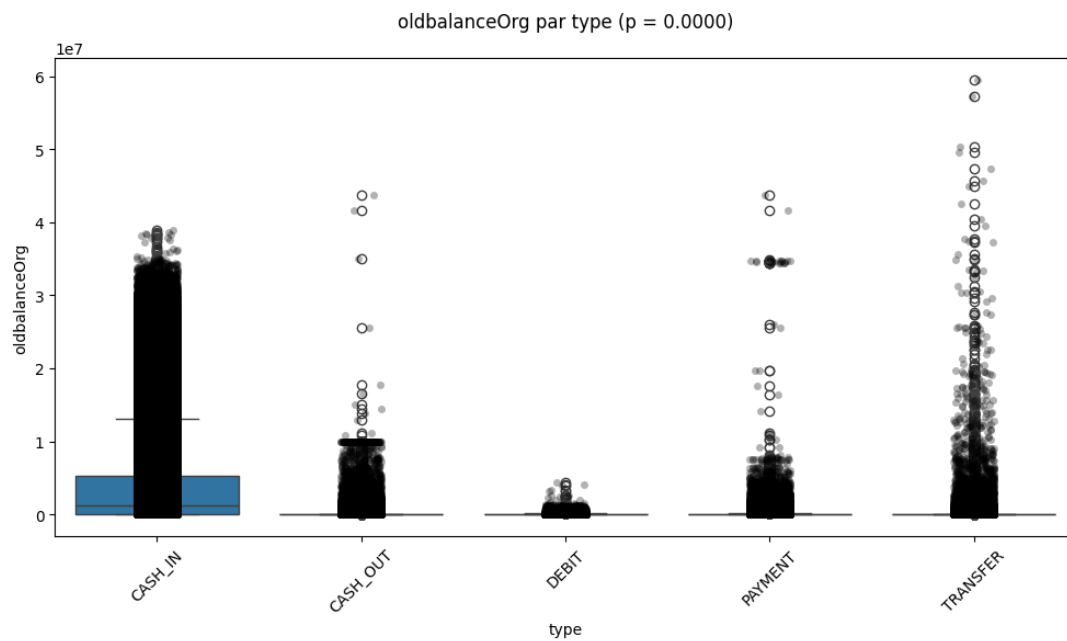
```
Test Kruskal-Wallis entre amount et type :  
H = 3906986.68, p = 0.0000 → Association significative
```

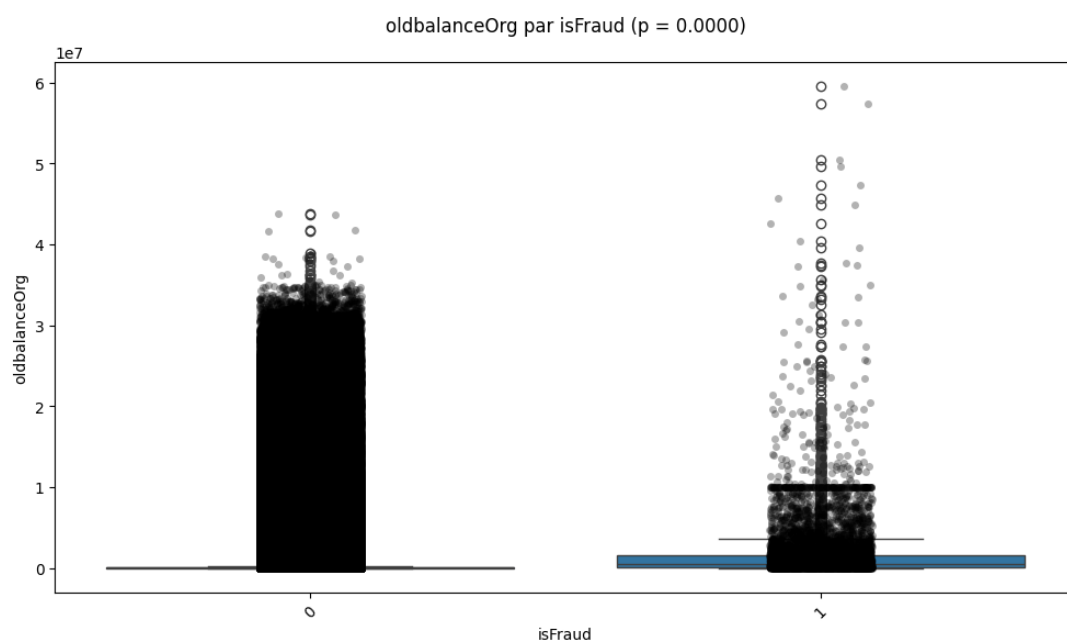
Test Kruskal-Wallis entre amount et isFraud :
 $H = 8273.37$, $p = 0.0000 \rightarrow$ Association significative



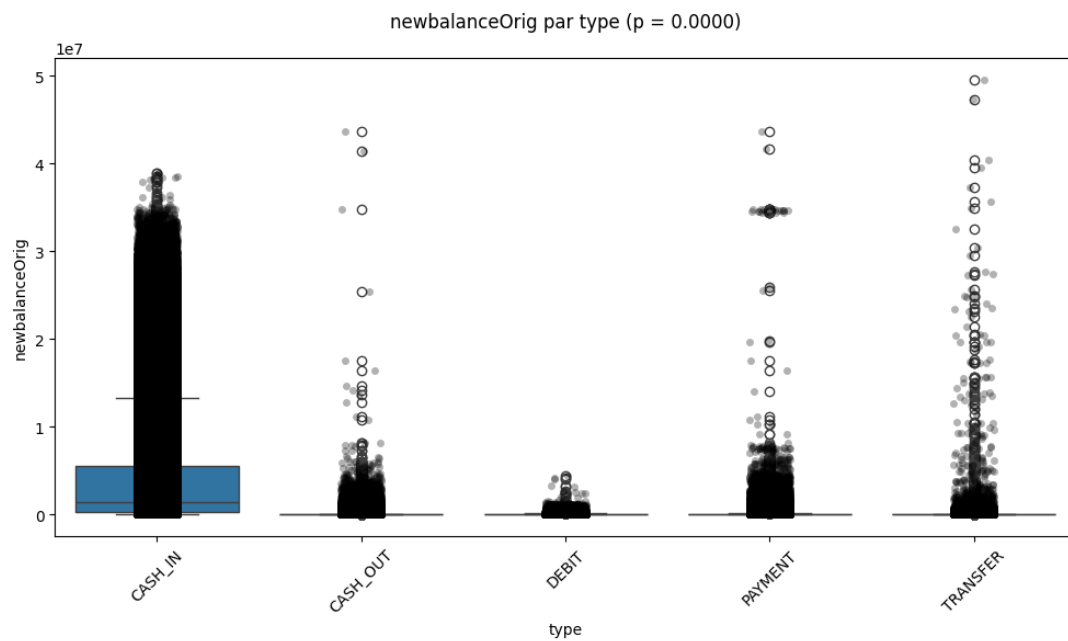
Test Kruskal-Wallis entre oldbalanceOrg et type :
 $H = 1868488.64$, $p = 0.0000 \rightarrow$ Association significative



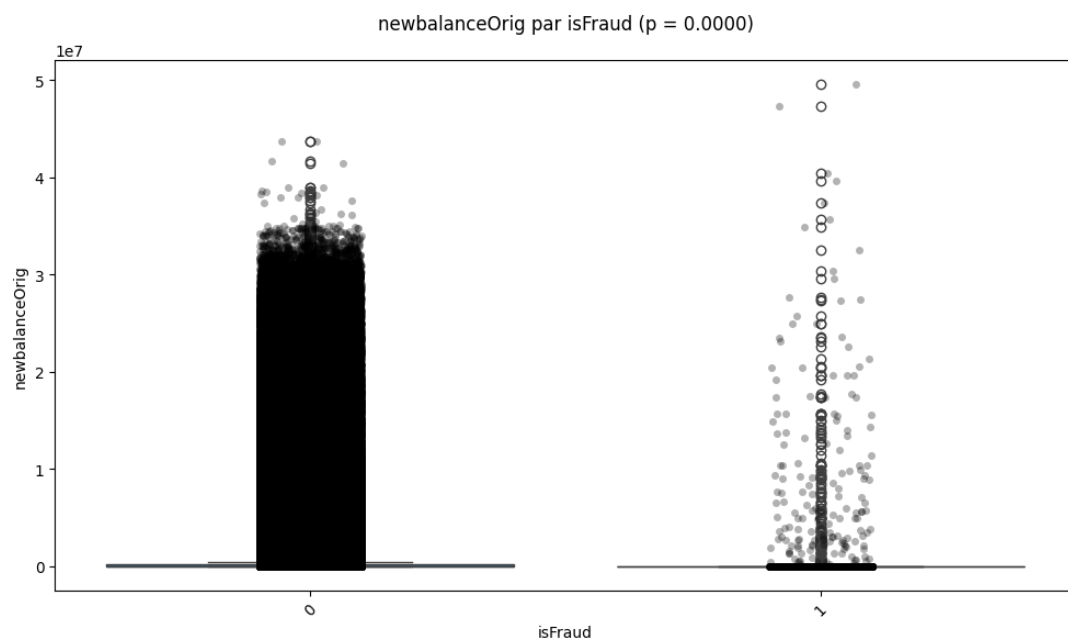
Test Kruskal-Wallis entre oldbalanceOrg et isFraud :
 $H = 9892.17$, $p = 0.0000 \rightarrow$ Association significative



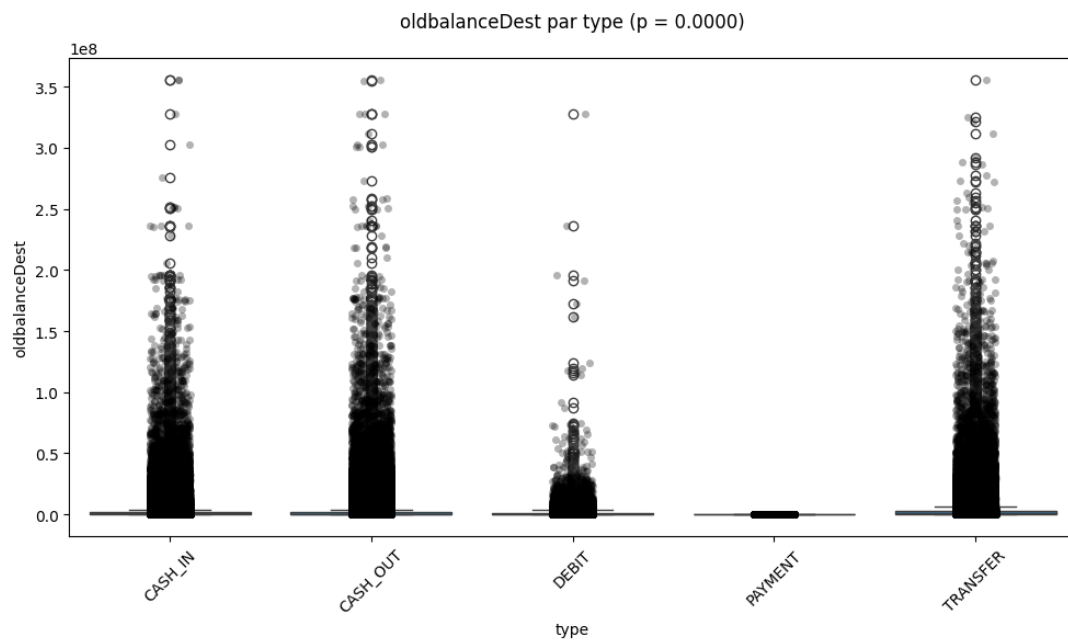
Test Kruskal-Wallis entre newbalanceOrig et type :
 $H = 4038804.73$, $p = 0.0000 \rightarrow$ Association significative



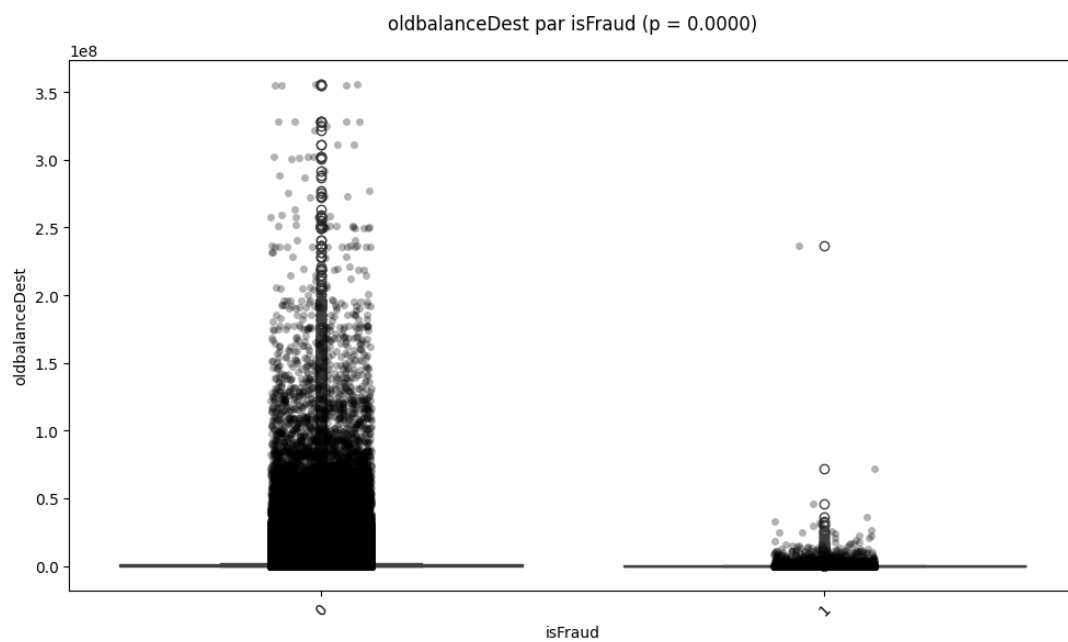
Test Kruskal-Wallis entre newbalanceOrig et isFraud :
 $H = 4999.38$, $p = 0.0000 \rightarrow$ Association significative



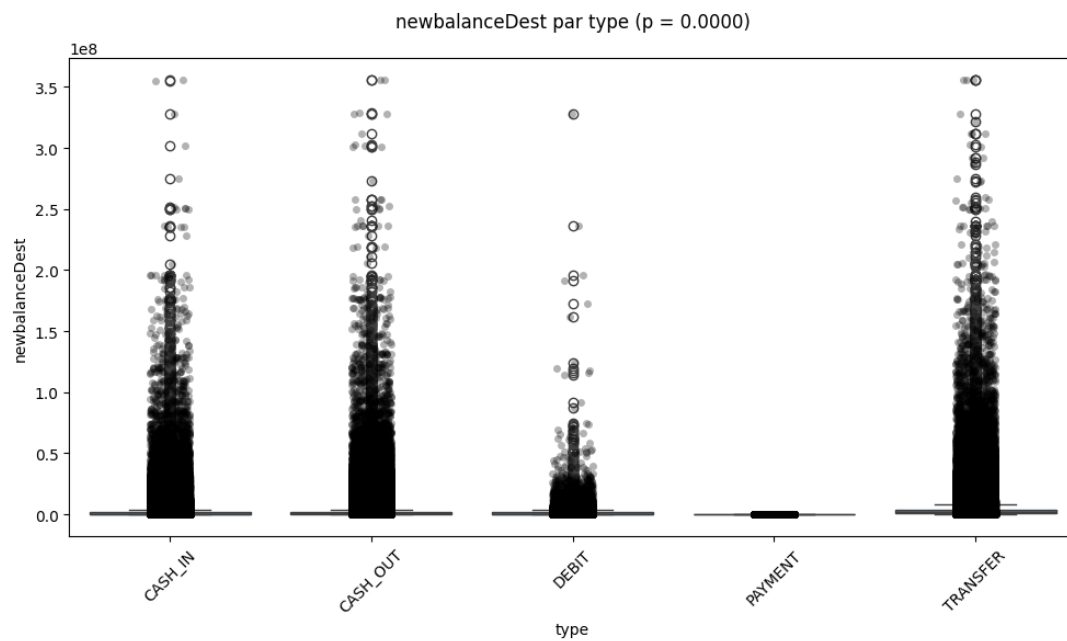
Test Kruskal-Wallis entre oldbalanceDest et type :
 $H = 3517554.62$, $p = 0.0000 \rightarrow$ Association significative



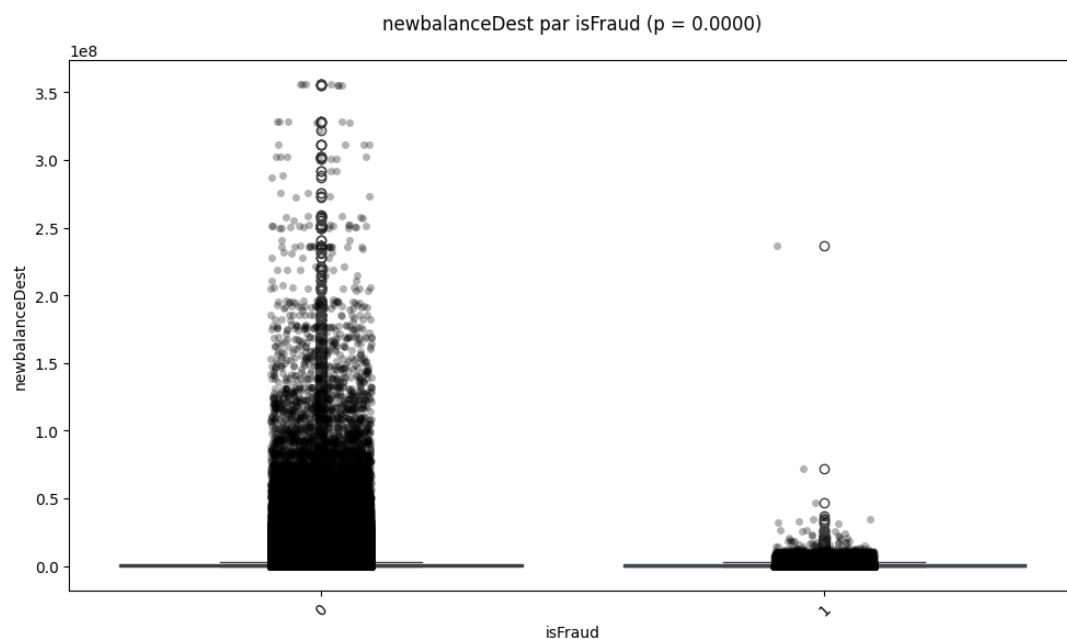
Test Kruskal-Wallis entre oldbalanceDest et isFraud :
 $H = 1869.46$, $p = 0.0000 \rightarrow$ Association significative



Test Kruskal-Wallis entre newbalanceDest et type :
 $H = 4170791.50$, $p = 0.0000 \rightarrow$ Association significative



Test Kruskal-Wallis entre newbalanceDest et isFraud :
 $H = 170.84$, $p = 0.0000 \rightarrow$ Association significative



Interprétations

Ces nouvelles variables comme : DiffSoldeOrig et DiffSoldeDest (qui vérifient si l'argent est vraiment retiré du compte client. Si 0 $\rightarrow$ anomalie possible.), détectent des anomalies fréquentes, notamment dans des cas de fraude (ex. : argent "disparaît" ou "apparaît"). La variable is-

SameAccount (qui permet de détecter si le compte de départ est égal au compte d'arrivée) = 0 pour tous : donc pas discriminant ici.

activiteAnormale → valeur 2 indique une double anomalie (origine + destination), très utile.

Ces variables renforcent le signal de comportements soupçonnés frauduleux.

4.2. Analyse Multivariée après le Feature Engineering

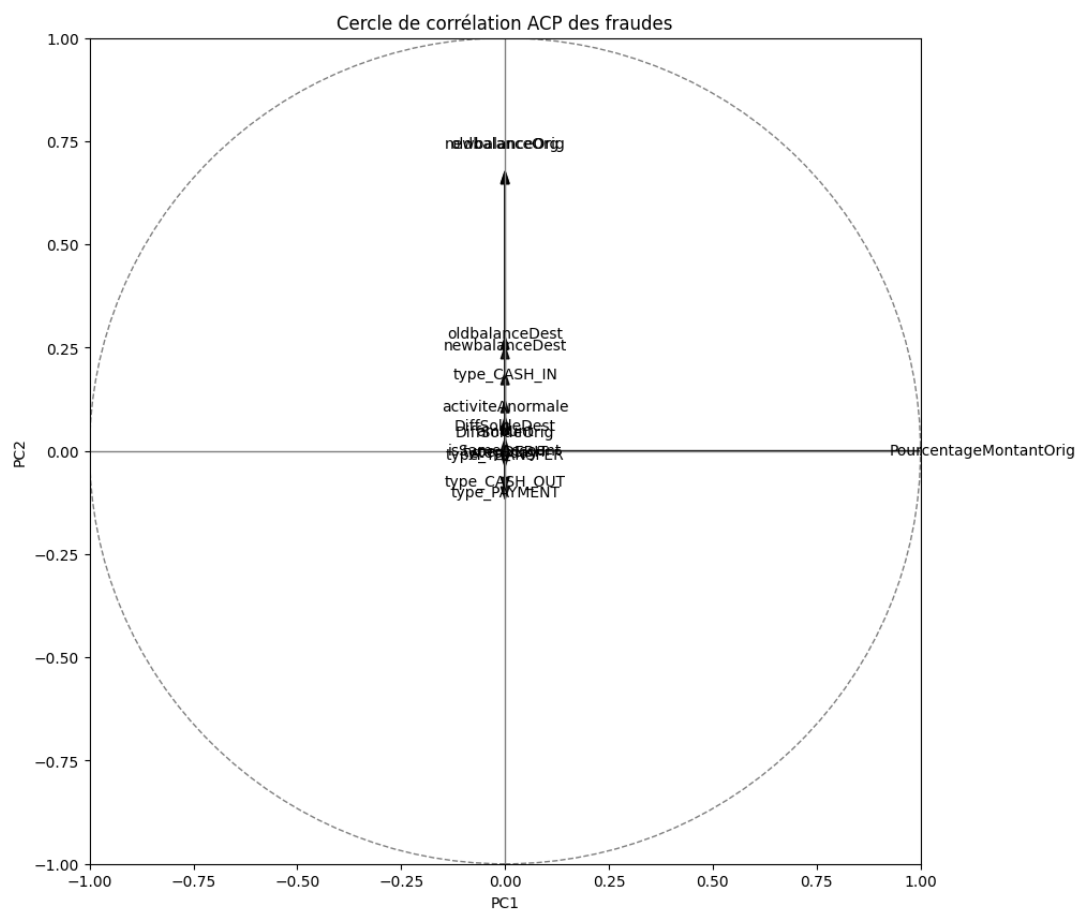
- Encodage
- Normalisation
- Définition du target et des features
- Instanciation de PCA

```
[10]: # Encoding des variables catégorielles
df_demmmies = pd.get_dummies(data_1[["type"]], prefix=["type"],
    ↪ prefix_sep="_", dtype=int)
df = pd.concat([data_1,df_demmmies], axis=1)
df.drop(columns=["type"], inplace = True)

# Normalisation des variables
df[[
    'amount', 'oldbalanceOrig', 'newbalanceOrig',
    'oldbalanceDest', 'newbalanceDest', 'step_day']] =
    ↪ (df[['amount', 'oldbalanceOrig', 'newbalanceOrig', 'oldbalanceDest',
    ↪ 'newbalanceDest', 'step_day']] - df[['amount', 'oldbalanceOrig',
    ↪ 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest',
    'step_day']].mean())/df[['amount', 'oldbalanceOrig',
    ↪ 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest', 'step_day']].std()
df_vf = df.copy()
```

```
[13]: an_after = MultivariateAnalyser(df_vf)
an_after.lancer_analyse()
```

--Analyse Multivariée --



-- Les Composants Principaux --

	Dimension	Valeur propre	Taux_variance	Taux_variance_cum
0	PCA_0	2.584952e+11	1.0	1.0
1	PCA_1	2.147000e+00	0.0	1.0
2	PCA_2	1.676000e+00	0.0	1.0
3	PCA_3	1.000000e+00	0.0	1.0
4	PCA_4	5.200000e-01	0.0	1.0
5	PCA_5	3.620000e-01	0.0	1.0
6	PCA_6	3.000000e-01	0.0	1.0
7	PCA_7	1.460000e-01	0.0	1.0
8	PCA_8	1.070000e-01	0.0	1.0
9	PCA_9	6.200000e-02	0.0	1.0
10	PCA_10	8.000000e-03	0.0	1.0
11	PCA_11	7.000000e-03	0.0	1.0
12	PCA_12	1.000000e-03	0.0	1.0
13	PCA_13	0.000000e+00	0.0	1.0
14	PCA_14	0.000000e+00	0.0	1.0
15	PCA_15	0.000000e+00	0.0	1.0

-- Influence de chaque variable dans les composants --

	PCA_1	PCA_2	PCA_3 \
PourcentageMontantOrig	1.000000e+00	-2.979783e-07	-1.122062e-06

amount	1.607082e-06	4.219399e-02	6.515435e-02
newbalanceDest	8.605063e-07	2.231253e-01	6.430570e-01
oldbalanceDest	6.054529e-07	2.463096e-01	6.727538e-01
type_TRANSFER	1.374247e-07	-8.152279e-03	2.428221e-02
DiffSoldeOrig	4.076303e-08	4.004814e-02	1.749633e-02
step_day	2.695479e-08	-4.222665e-03	5.271563e-02
type_CASH_OUT	9.167774e-09	-6.401319e-02	8.339555e-02
isSameAccount	0.000000e+00	0.000000e+00	-4.336809e-19
type_DEBIT	-1.756655e-09	-1.142765e-03	1.383769e-03
activiteAnormale	-2.757388e-08	9.262916e-02	-4.443263e-02
type_CASH_IN	-5.863893e-08	1.602958e-01	-4.004766e-02
DiffSoldeDest	-6.833692e-08	5.258101e-02	-6.192896e-02
oldbalanceOrg	-7.893704e-08	6.463769e-01	-2.310380e-01
newbalanceOrig	-7.981877e-08	6.477419e-01	-2.311900e-01
type_PAYMENT	-8.619693e-08	-8.698759e-02	-6.901387e-02

	PCA_4	PCA_5	PCA_6 \
PourcentageMontantOrig	8.515715e-09	2.214863e-07	-1.206699e-06
amount	1.083838e-02	-1.447630e-01	7.732642e-01
newbalanceDest	-3.537313e-02	4.759671e-02	3.080972e-03
oldbalanceDest	-3.538381e-02	8.833983e-02	-1.419272e-01
type_TRANSFER	5.794658e-04	-4.012032e-02	1.285490e-01
DiffSoldeOrig	-5.442055e-03	2.781569e-02	2.975922e-01
step_day	9.973013e-01	3.697963e-02	3.839538e-03
type_CASH_OUT	-9.202003e-03	-4.641098e-01	-6.702067e-05
isSameAccount	-0.000000e+00	5.551115e-17	1.665335e-16
type_DEBIT	3.095739e-04	-5.751520e-03	-8.258585e-03
activiteAnormale	-3.356419e-02	5.488147e-01	3.321526e-01
type_CASH_IN	4.262732e-03	1.358551e-01	2.092126e-01
DiffSoldeDest	-2.812213e-02	5.209990e-01	3.456034e-02
oldbalanceOrg	1.903305e-02	-1.048721e-01	-8.204950e-02
newbalanceOrig	1.844893e-02	-8.791784e-02	-7.868278e-02
type_PAYMENT	4.050231e-03	3.741266e-01	-3.294359e-01

	PCA_7	PCA_8	PCA_9 \
PourcentageMontantOrig	-8.813593e-07	3.000549e-07	1.111743e-07
amount	5.340445e-01	-1.930527e-01	-1.449007e-01
newbalanceDest	8.330664e-02	-6.111383e-02	-5.068496e-03
oldbalanceDest	-3.539092e-02	4.496560e-02	1.523316e-02
type_TRANSFER	1.270417e-01	1.704198e-01	5.178328e-01
DiffSoldeOrig	-4.110611e-01	-1.570359e-01	5.137464e-01
step_day	-2.592055e-02	-1.398867e-02	-9.837192e-03
type_CASH_OUT	-3.921551e-01	-4.789995e-01	-3.712326e-01
isSameAccount	-2.220446e-16	-2.255141e-16	-2.775558e-16
type_DEBIT	8.636374e-03	2.430632e-02	-3.122653e-03
activiteAnormale	-4.000289e-01	-2.704984e-01	6.675694e-02
type_CASH_IN	-1.629102e-01	6.562209e-01	-2.865837e-01
DiffSoldeDest	1.103221e-02	-1.134624e-01	-4.469895e-01
oldbalanceOrg	5.646344e-02	-1.159023e-01	3.534305e-02
newbalanceOrig	3.468043e-02	-6.779664e-02	2.499766e-02
type_PAYMENT	4.193872e-01	-3.719474e-01	1.431061e-01

	PCA_10	PCA_11	PCA_12	\
PourcentageMontantOrig	8.334211e-08	-4.122009e-09	1.967199e-08	
amount	-1.032256e-01	-1.383476e-02	-1.431641e-01	
newbalanceDest	-3.789028e-03	1.073283e-01	7.147940e-01	
oldbalanceDest	-1.683341e-02	-1.014861e-01	-6.659168e-01	
type_TRANSFER	6.410180e-01	-2.370497e-01	2.529606e-02	
DiffSoldeOrig	-3.426591e-01	1.947270e-02	1.728121e-03	
step_day	1.549176e-02	5.442266e-04	2.523842e-03	
type_CASH_OUT	8.025424e-02	-2.137766e-01	1.007356e-02	
isSameAccount	2.220446e-16	-7.910339e-16	9.992007e-16	
type_DEBIT	1.559426e-02	8.840285e-01	-1.320504e-01	
activiteAnormale	7.418193e-02	1.616893e-02	-6.649165e-03	
type_CASH_IN	-3.372730e-01	-2.223210e-01	8.083643e-02	
DiffSoldeDest	4.168410e-01	-3.303769e-03	-8.377287e-03	
oldbalanceOrg	3.316732e-02	5.911184e-03	-3.930155e-03	
newbalanceOrig	4.001879e-03	-2.474776e-03	4.990598e-03	
type_PAYMENT	-3.995935e-01	-2.108812e-01	1.584434e-02	

	PCA_13	PCA_14	PCA_15	PCA_16
PourcentageMontantOrig	-2.040861e-08	4.004389e-18	-4.775073e-17	1.391540e-17
amount	1.230320e-02	-2.402625e-12	2.865067e-11	-8.349182e-12
newbalanceDest	-1.365884e-03	2.091949e-13	-2.447885e-12	7.128204e-13
oldbalanceDest	3.875573e-04	-2.716058e-14	2.765272e-13	-8.011291e-14
type_TRANSFER	1.180055e-02	3.446917e-02	4.284263e-01	-1.235425e-01
DiffSoldeOrig	-5.280230e-03	5.755395e-01	-3.986948e-02	2.231809e-02
step_day	2.809075e-04	-5.834730e-14	6.188866e-13	-1.803849e-13
type_CASH_OUT	1.350673e-02	3.446917e-02	4.284263e-01	-1.235425e-01
isSameAccount	-4.195949e-14	-1.795546e-02	2.783625e-01	9.603082e-01
type_DEBIT	5.776088e-03	3.446917e-02	4.284263e-01	-1.235425e-01
activiteAnormale	-2.284983e-03	-5.755395e-01	3.986948e-02	-2.231809e-02
type_CASH_IN	-3.803542e-02	3.446917e-02	4.284263e-01	-1.235425e-01
DiffSoldeDest	2.995248e-03	5.755395e-01	-3.986948e-02	2.231809e-02
oldbalanceOrg	-7.012166e-01	1.379069e-10	-1.645968e-09	4.796614e-10
newbalanceOrig	7.115122e-01	-1.397585e-10	1.668058e-09	-4.860987e-10
type_PAYMENT	6.952047e-03	3.446917e-02	4.284263e-01	-1.235425e-01

FIN DE L'ANALYSE MULTIVARIEE

Interprétation des résultats

Les nouvelles variables DiffSoldeOrig, PourcentageMontantOrig, activiteAnormale répartissent mieux la variance, apparaissent donc comme contributives (visibles dans le cercle de corrélation), confirmant leur pertinence. → information mieux structurée. Les nouvelles variables dérivées révèlent une structure utile. PCA améliorée: Meilleure répartition de la variance sur plusieurs composantes, mais toujours dominée par quelques-unes (PCA_0 à PCA_2).

XGBoost peut exploiter ces non-linéarités mieux que des modèles linéaires classiques.

Instancier le modèle

```
[11]: # Définition du train et test
features = df_vf.drop(columns="isFraud")
target = df_vf["isFraud"]
X_train,X_test, y_train, y_test = train_test_split(features,target,
→train_size=0.3, random_state=42)

[15]: # Instancier le modèle
model_xgb = XGBClassifier(
    n_estimators = 100,
    n_jobs = -1,
    learning_rate = 0.1,
    max_depth = 6,
    subsample = 0.8,
    enable_categorical = True)
# Entraînement du modèle
model_xgb.fit(X_train,y_train)

# Prediction du modèle
y_pred = model_xgb.predict(X_test)

[4]: def EvaluationModele(y_true, y_pred):
    # Matrice de confusion
    print(f"\n La matrice de confusion \n")
    matrice_confusion = tabulate(pd.DataFrame(
        confusion_matrix(y_true=y_true, y_pred=y_pred)),
        headers=["Faux Negatif", "Vraie Negatif"],tablefmt = "rounded_grid")
    print(matrice_confusion)

    # Rapport d'évaluation du modèle
    print(f"\n Les indicateurs de performances \n")
    print(classification_report(y_test, y_pred))
```

Pipeline du Modèle

```
[12]: # Définition du train et test
X = data_1.drop(columns="isFraud")
y = data_1["isFraud"]
X_train,X_test, y_train, y_test = train_test_split(X,y, train_size=0.3,
↳random_state=42)

# Pretraitement des données
preprocessing = ColumnTransformer([
    ("cat",OneHotEncoder(handle_unknown='ignore'),['type']),
    ("norm", StandardScaler(),['amount','oldbalanceOrg',
↳'newbalanceOrig','oldbalanceDest', 'newbalanceDest','step_day'])
],remainder='passthrough')

# Implementation du pipeline
pipe_xgboost = Pipeline([
    ('prepros', preprocessing),
    ('model',XGBClassifier(
        n_estimators = 100,
        n_jobs = -1,
        learning_rate = 0.1,
        max_depth = 6,
        subsample = 0.8,
        enable_categorical = True))
])

# Entraînement final
pipe_xgboost.fit(X_train, y_train)
```

```
[12]: Pipeline(steps=[('prepros',
                        ColumnTransformer(remainder='passthrough',
                        transformers=[('cat',
OneHotEncoder(handle_unknown='ignore'),
                        ['type']),
                        ('norm', StandardScaler(),
                        ['amount', 'oldbalanceOrg',
                        'newbalanceOrig',
                        'oldbalanceDest',
                        'newbalanceDest',
                        'step_day'])])),
                    ('model',
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, col...
              feature_types=None, gamma=None,
↳grow_policy=None,
              importance_type=None,
              interaction_constraints=None, learning_rate=0.
↳1,
              max_bin=None, max_cat_threshold=None,
```

```

max_cat_to_onehot=None, max_delta_step=None,
max_depth=6, max_leaves=None,
min_child_weight=None, missing=nan,
monotone_constraints=None, multi_strategy=None,
n_estimators=100, n_jobs=-1,
num_parallel_tree=None, random_state=None,
...)))]

```

```

[18]: # Prediction du modèle
y_predite = pipe_xgboost.predict(X_test)

```

```

[19]: # Evaluation du modèle
EvaluationModele(y_test,y_predite)

```

La matrice de confusion

	Faux Negatif	Vraie Negatif
0	4.44803e+06	36
1	29	5742

Les indicateurs de performances

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4448063
1	0.99	0.99	0.99	5771
accuracy			1.00	4453834
macro avg	1.00	1.00	1.00	4453834
weighted avg	1.00	1.00	1.00	4453834

Interprétation

Malgré le ratio 1:1000, le modèle capture 99% des fraudes sans sacrifier la précision.

L'importance de chaque variables dans le resultat de la prediction du modèle

```

[13]: # Extraire le preprocessor
preprocessor = pipe_xgboost.named_steps["prepros"]

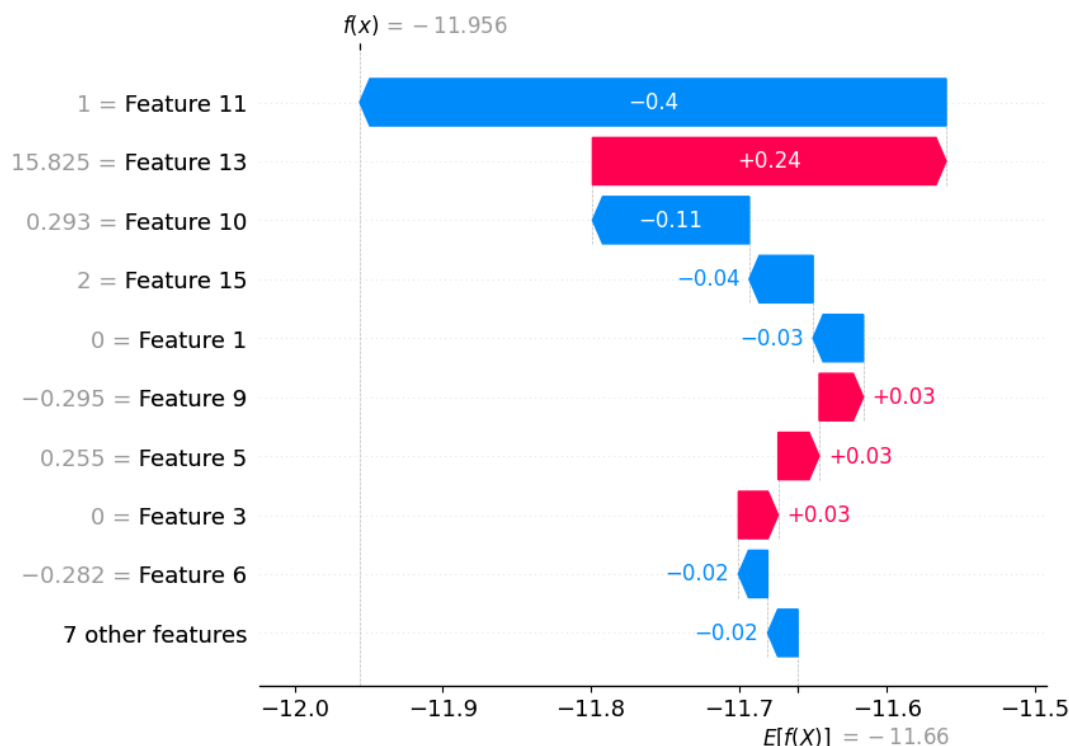
# Transformer X_test pour qu'il soit numérique
X_transformed = preprocessor.transform(X_test)

# Utiliser X_transformed sur le modèle XGBoost
model = pipe_xgboost.named_steps["model"]
explainer = shap.Explainer(model, X_transformed)

```

```
shap_values = explainer(X_transformed)
shap.plots.waterfall(shap_values[0])
```

100%|=====| 4453576/4453834 [215:29<00:00]



Interprétation

Les variables telles que type_CASH_IN et type_DEBIT (Feature 11/13) dominent, aligné avec les résultats ACP.

type_TRANSFER/type_CASH_OUT contribuent significativement (confirmant l'analyse bi-variée).

Le waterfall plot montre comment chaque feature influence la prédiction pour un cas individuel, utile pour l'audit

Perspectives

Bien que le modèle prédictif basé sur le machine learning ait montré d'excellentes performances pour identifier les transactions suspectes individuellement, une prochaine étape incontournable sera d'aller **au-delà des observations isolées**, en analysant les **relations entre les entités** : comptes, bénéficiaires, transactions récurrentes.

Pour cela, la **théorie des graphes** s'impose comme une approche stratégique. En représentant les transactions comme des arêtes reliant des nœuds (comptes émetteurs et récepteurs), il devient possible de :

- Identifier des **comportements collaboratifs suspects** (fraude en réseau),
- Détecter des **comptes pivot ou transitaires**,
- Reconnaître des **motifs cycliques ou en étoile**, typiques du blanchiment d'argent.

Ainsi, l'intégration d'un moteur d'analyse graphes (avec des outils comme 'networkx', 'Neo4j', ou 'GraphSAGE') permettrait **d'optimiser le système de détection actuel** et d'augmenter encore sa robustesse face à des scénarios de fraude complexes.